

北京邮电大学

本科毕业设计（论文）



题目：基于 Web 的数字网一体化资源调度管理系统
设计与实现

姓 名	汪洪申
学 院	信息与通信工程学院
专 业	通信工程
班 级	2022211116
学 号	2021210448
指导教师	胡鹤飞

2026 年 5 月

北 京 邮 电 大 学

本科毕业设计（论文）诚信声明

本人声明所呈交的毕业设计（论文），题目《基于 Web 的数字阵一体化资源调度管理系统设计与实现》是本人在指导教师的指导下，独立进行研究工作所取得的成果。尽我所知，除了文中特别加以标注和致谢中所罗列的内容以外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得北京邮电大学或其他教育机构的学位或证书而使用过的材料。

申请学位论文与资料若有不实之处，本人承担一切相关责任。

本人签名：_____ 日期：_____

关于论文使用授权的说明

本人完全了解并同意北京邮电大学有关保留、使用学位论文的规定，即：北京邮电大学拥有以下关于学位论文的无偿使用权，具体包括：学校有权保留并向国家有关部门或机构送交学位论文，有权允许学位论文被查阅和借阅；学校可以公布学位论文的全部或部分内容，有权允许采用影印、缩印或其它复制手段保存、汇编学位论文，将学位论文的全部或部分内容编入有关数据库进行检索。（保密的学位论文在解密后遵守此规定）

本人签名：_____ 日期：_____

导师签名：_____ 日期：_____

基于 Web 的数字阵一体化资源调度管理系统设计与实现

摘要

随着数字阵系统任务规模和并发程度不断提高，任务执行过程通常同时依赖天线阵元、CPU 与 GPU 等多类异构资源。传统以静态台账和人工配置为主的资源管理方式难以满足多任务场景下的统一建模、联合调度和过程追踪需求，容易导致资源冲突、利用率不足以及调度过程不可解释等问题。针对上述问题，本文设计并实现了一套基于 Web 的数字阵一体化资源调度管理系统，旨在提升数字阵任务资源管理的统一性、调度过程的自动化程度和运行状态的可观测性。

系统采用前后端分离架构，后端基于 Spring Boot 多模块结构实现认证鉴权、资源管理、任务管理和调度控制，前端基于 React 与 Vite 实现任务提交、资源监控、调度结果展示和时间线可视化。系统围绕“资源—任务—调度过程”构建数据模型，将天线阵元、CPU 节点、GPU 资源及其分配记录纳入统一管理，并通过任务表、资源表、分配表和调度日志表支撑任务提交、资源分配、状态反馈和结果追踪的完整流程。

在调度机制方面，系统以任务需求为输入，综合考虑阵元数量、CPU 核数、GPU 显存、任务优先级、截止时间以及依赖关系等约束，自动识别低时延任务、多资源均衡任务、GPU 辅助任务、CPU 密集型任务和依赖约束任务等典型场景，并据此选择不同的内部调度路径。针对阵元资源，系统支持单阵面优先和跨阵面扩展分配；针对计算资源，系统支持紧凑式 CPU 分配、负载均衡式 CPU 分配和 GPU 单卡局部性分配。同时，系统通过调度日志、剩余时间写回和时间片反馈机制，对任务运行、等待、更新和完成释放过程进行持续记录，提升调度过程的可解释性。

在系统实现基础上，本文构建了多类典型测试场景，对系统的场景识别、联合资源分配和等待处理能力进行了验证。实验结果表明，系统能够根据任务特征选择差异化调度路径，在 CPU 密集型场景下实现多节点负载均衡，在 GPU 场景下实现单卡局部性分配，在依赖不满足场景下正确阻塞任务并记录等待原因。本文实现的系统完成了数字阵资源从静态管理向任务驱动的动态联合调度管理的转变，为数字阵任务资源协同调度提供了一种具有工程可行性的实现方案。

关键词 数字阵 资源调度 联合调度 异构资源 Web 系统

Design and Implementation of a Web-Based Integrated Resource Scheduling and Management System for Digital Arrays

ABSTRACT

With the increasing scale and concurrency of tasks in digital array systems, task execution often depends on multiple types of heterogeneous resources, including antenna elements, CPU nodes, and GPU resources. Traditional resource management methods, which mainly rely on static resource records and manual configuration, are insufficient for unified modeling, coordinated scheduling, and process tracing in multi-task scenarios. These limitations may lead to resource conflicts, low utilization, and poor interpretability of scheduling decisions. To address these problems, this thesis designs and implements a Web-based integrated resource scheduling and management system for digital arrays, aiming to improve the consistency of resource management, the automation of scheduling, and the observability of task execution.

The system adopts a front-end and back-end separated architecture. The back end is implemented with a Spring Boot multi-module structure, providing authentication, resource management, task management, and scheduling control. The front end is implemented with React and Vite, supporting task submission, resource monitoring, scheduling result display, and timeline visualization. Centered on the relationship among resources, tasks, and scheduling processes, the system establishes a unified data model for antenna elements, CPU nodes, GPU resources, and allocation records. Task tables, resource tables, allocation tables, and scheduling log tables are used to support the complete workflow of task submission, resource allocation, state feedback, and result tracing.

In terms of scheduling mechanism, the system takes task requirements as input and comprehensively considers constraints such as the number of required antenna elements, CPU cores, GPU memory, task priority, deadline, and dependency relations. Based on these constraints, it automatically identifies typical scenarios including low-latency tasks, balanced multi-resource tasks, GPU-assisted tasks, CPU-intensive tasks, and dependency-constrained tasks, and then selects different internal scheduling paths accordingly. For antenna resources, the system supports single-surface priority allocation and cross-surface expansion allocation. For computing resources, it supports packed CPU allocation, load-balanced CPU allocation,

and single-card GPU locality allocation. Meanwhile, scheduling logs, remaining-time write-back, and time-slice feedback mechanisms are introduced to continuously record task running, waiting, updating, and completion-release processes, thereby improving the interpretability of scheduling behavior.

Based on the implemented system, several typical test scenarios are constructed to verify the system's capabilities in scenario identification, joint resource allocation, and waiting-state handling. The experimental results show that the system can select differentiated scheduling paths according to task characteristics. It achieves multi-node load balancing in CPU-intensive scenarios, single-card locality allocation in GPU-related scenarios, and correct task blocking with recorded waiting reasons when dependencies are not satisfied. The implemented system realizes the transformation from static resource management to task-driven dynamic joint scheduling, providing an engineering-feasible solution for coordinated resource scheduling in digital array systems.

KEY WORDS digital array resource scheduling heterogeneous resource Web-based system

目录

第一章 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	2
1.2.1 数字阵与相控阵任务调度研究	2
1.2.2 多资源联合调度与集群资源管理研究	2
1.2.3 异构计算资源调度与 Web 可视化管理研究	3
1.2.4 现有研究不足分析	4
1.3 研究内容与创新点	4
1.4 章节安排	6
第二章 系统总体设计	9
2.1 系统需求分析	9
2.2 非功能需求分析	10
2.3 系统总体架构	11
2.4 系统业务流程	13
2.5 数据库设计	14
2.6 引言	15
2.7 本章小结	17
第三章 天线/CPU/GPU 联合调度设计	19
3.1 联合调度问题分析	19
3.2 天线阵元调度设计	20
3.2.1 阵元候选过滤与冲突检测	21
3.2.2 多策略阵元搜索算法	21
3.2.3 调度策略选择机制	22
3.3 CPU/GPU 计算资源调度设计	23
3.3.1 CPU 资源分配策略	23
3.3.2 GPU 资源分配策略	24
3.3.3 计算资源原型算法与系统落地	25
3.3.4 计算资源调度结果处理	25
3.4 联合调度流程与主循环	26
3.5 本章小结	31
第四章 任务场景识别与自动决策机制	33

4.1 设计思路	33
4.2 场景识别规则	34
4.3 自动决策映射	36
4.4 决策结果输出	38
4.5 本章小结	39
第五章 调度日志与动态反馈设计	41
5.1 调度日志设计	41
5.2 时间片与状态反馈机制	42
5.3 前端可视化展示	44
5.4 本章小结	48
第六章 系统测试与结果分析	49
6.1 测试目的与测试环境	49
6.2 测试场景设计	50
6.3 测试结果与分析	52
6.4 调度算法性能量化对比	54
6.4.1 天线调度算法执行时间对比	54
6.4.2 天线调度算法可行性对比	55
6.4.3 天线调度算法解质量对比	56
6.4.4 天线调度算法综合评价	58
6.4.5 CPU 调度模式负载均衡对比	58
6.4.6 多任务并发调度性能验证	59
6.4.7 算法可扩展性验证	60
6.5 系统可用性与不足分析	62
6.6 本章小结	63
第七章 结论与展望	65
7.1 研究结论	65
7.2 不足之处	66
7.3 展望	67
参考文献	69
致谢	71
附录 1 社会、健康、安全、法律、文化以及环境等非技术因素影响评估	73

第一章 绪论

1.1 研究背景与意义

随着雷达探测、卫星通信、电子对抗和复杂信号处理等领域的发展，数字阵系统在现代信息系统中的应用越来越广泛。相比传统机械扫描或单通道处理方式，数字阵系统具有阵元规模大、波束形成灵活、响应速度快和多任务并发能力强等特点，能够为多目标探测、多波束通信和实时信号处理等任务提供支撑。因此，如何对数字阵系统中的各类资源进行高效管理和合理调度，已经成为系统工程化应用中的重要问题。

在实际运行过程中，一个数字阵任务通常不仅需要占用一定数量的天线阵元，还可能同时依赖 CPU 计算资源和 GPU 等加速资源。不同任务在阵元数量、计算能力、运行时长、优先级和截止时间等方面存在差异。当多个任务并发执行时，系统需要在多类异构资源之间进行协调分配。如果缺乏统一的资源建模和调度机制，容易出现资源状态不清晰、任务分配冲突、关键任务响应不及时以及资源利用率不足等问题。

目前，部分数字阵或实验平台中的资源管理仍以静态台账、人工配置或单独模块管理为主。这类方式在任务规模较小时可以满足基本需求，但在多任务并发和资源状态动态变化的场景下存在明显不足。一方面，天线阵元、CPU 和 GPU 等资源缺乏统一表达，难以形成面向任务需求的整体资源视图；另一方面，调度过程依赖人工经验，缺少对任务等待原因、资源分配过程和任务状态变化的持续记录，导致系统运行过程不够透明，后续分析和复盘也较为困难。

因此，设计并实现一套基于 Web 的数字阵一体化资源调度管理系统具有较强的现实意义。通过对天线阵元、CPU 和 GPU 等资源进行统一建模，并结合任务需求进行联合调度，可以提升资源分配的合理性和任务响应能力；通过引入调度日志、状态反馈和可视化展示，可以增强调度过程的可解释性和系统运行的可观测性。该课题不仅有助于降低人工管理成本、提升资源利用率，也能够综合体现资源建模、调度策略设计、前后端系统实现和实验验证等毕业设计要求。

1.2 国内外研究现状

1.2.1 数字阵与相控阵任务调度研究

数字阵系统和相控阵雷达系统通常具有阵元数量多、任务类型复杂、资源约束强和实时性要求高等特点，因此相关研究较早关注雷达任务调度、波束驻留安排和阵列资源利用等问题。在相控阵雷达任务调度方面，已有研究通常围绕任务优先级、截止期、时间窗和任务收益等因素展开。例如，张浩为等针对防空相控阵雷达负载饱和情况下的任务调度问题，提出了结合目标威胁度和截止期的动态优先级调度算法，用于提高雷达系统在高负载条件下的任务执行能力^[1]。相关研究表明，相控阵雷达调度通常需要综合考虑任务优先级确定和调度策略选择两个方面，单纯依赖某一参数进行排序难以满足复杂任务环境下的调度需求。张浩为等在后续工作中进一步研究了数字阵列雷达在线交错调度问题，提出了面向多任务交错场景的在线调度算法^[2]。

在数字阵列系统研究方面，已有工作更多关注阵列波束形成、宽带多波束发射和阵列信号处理等底层技术。例如，朱元江等对数字阵列宽带多波束发射进行了设计与验证，说明数字阵列系统能够通过数字化方式实现更加灵活的多波束形成能力^[3]。这类研究为数字阵系统的硬件能力和信号处理能力提供了重要支撑，但其重点通常集中在阵列处理算法和波束形成机制本身，对上层任务如何占用阵元、如何与计算资源协同分配、如何进行可视化管理等问题讨论相对较少。

此外，在多雷达协同和任务规划方向，也有研究从任务效用最大化角度对多任务实时规划进行建模。例如，袁野等针对多雷达协同场景下的多任务实时规划问题，提出了基于任务效用最大化的在线任务规划模型^[4]。这说明雷达与阵列系统中的任务调度已经逐渐从单一任务排序转向多任务协同优化。但总体来看，现有数字阵或雷达调度研究更多面向波束、时间片、任务收益和雷达工作方式等专业场景，对于“天线阵元资源—CPU/GPU 计算资源—Web 管理系统”之间的一体化联合调度研究仍然不足。

1.2.2 多资源联合调度与集群资源管理研究

在通用计算领域，多资源联合调度已经形成了较多经典研究成果。其中，Ghodsi 等提出的主导资源公平算法（Dominant Resource Fairness, DRF）是多资源公平分配领域的代表性工作^[5]。该方法将传统最大最小公平思想推广到 CPU、内存等多种资源类型，通过比较用户在不同资源维度上的主导份额来实现多资源公平分配。DRF 的提出说明，在

多任务共享系统中，仅考虑单一资源维度往往无法保证整体公平性，多资源调度需要同时关注不同资源之间的占用比例。

在工程系统方面，Mesos 是较早用于数据中心资源共享的代表性平台。Mesos 通过在多个计算框架之间共享集群资源，使 Hadoop、MPI 等不同类型的计算框架能够在同一集群中运行，从而提升资源利用效率^[6]。Google 的 Borg、Omega 和 Kubernetes 也为大规模集群资源管理提供了重要实践经验，其中 Borg 面向生产服务和批处理任务的统一管理，Omega 探索更加开放的调度架构，而 Kubernetes 则进一步推动了容器编排、服务部署和资源管理的平台化发展^[7]。Kubernetes 官方也将其定义为用于自动化部署、扩展和管理容器化应用的开源系统，其调度器会根据资源可用性和约束条件为工作负载选择运行节点^[8]。

这些研究和系统表明，统一资源抽象、任务队列管理、调度策略选择和状态反馈是现代资源管理系统的重要组成部分。然而，Kubernetes、Mesos 等平台主要面向通用计算集群，其资源模型通常以 CPU、内存、存储、网络等标准化资源为核心。对于数字阵系统而言，天线阵元具有阵面分布、空间位置、复用次数和跨阵面约束等特征，不能简单等同于普通计算节点。因此，通用集群调度思想可以为本文提供多资源调度、公平控制和任务管理方面的参考，但仍需要结合数字阵业务特征进行适配。

1.2.3 异构计算资源调度与 Web 可视化研究

随着 GPU、FPGA 等加速资源在计算系统中的广泛应用，异构计算资源调度也成为近年来的重要研究方向。相比单纯 CPU 集群，异构计算环境中不同资源的计算能力、使用方式和分配粒度差异较大，调度系统不仅需要判断资源总量是否满足任务需求，还需要考虑局部性、负载均衡和资源碎片等问题。在高性能计算领域，Slurm 是常见的开源集群管理与作业调度系统，能够为用户分配计算节点并管理作业队列，适用于不同规模的 Linux 集群^[9]。这类系统体现了作业提交、资源分配、队列管理和运行监控等机制在工程系统中的重要性。

在云原生和 AI 计算场景中，Volcano 等调度系统进一步面向批处理、大数据和 AI 训练任务提供统一调度能力。Volcano 官方文档指出，其支持 Kubernetes 原生工作负载调度，并面向 PyTorch、TensorFlow、Spark、Flink、Ray 等计算框架提供支持^[10]。其统一调度能力也体现出当前调度系统正在从单一资源分配向多类型工作负载统一管理方向发展。这些研究和系统对本文具有启发意义：数字阵调度系统同样需要处理不同类型资源和不同类型任务之间的匹配关系，并通过统一调度入口降低资源管理复杂度。

在 Web 管理与可视化方面，现代资源管理系统通常不再只提供后台调度逻辑，而是通过可视化界面展示资源状态、任务队列、运行结果和异常信息。对于多资源调度系统而言，前端可视化不仅是用户交互入口，也是调度过程解释的重要手段。尤其是在任务等待、资源不足、依赖未满足或任务完成释放等场景中，若系统能够通过日志、时间线和资源负载图展示调度过程，就可以降低系统使用门槛，提高调度行为的可理解性和可复盘性。因此，在本文系统设计中，Web 可视化不仅承担资源管理界面的作用，也承担调度过程解释和实验结果展示的作用。

1.2.4 现有研究不足分析

综合上述研究可以看出，现有工作已经分别在数字阵列技术、雷达任务调度、多资源公平分配、集群资源管理和异构计算调度等方面取得了较多成果。数字阵和相控阵领域的研究强调任务实时性、波束控制和任务优先级；通用集群调度研究强调多资源公平性、任务队列和资源利用率；异构计算调度研究则关注 CPU、GPU 等不同计算资源之间的协同分配。这些成果为本文提供了重要理论和工程参考。

但是，现有研究仍存在以下不足。首先，数字阵相关研究多集中在波束形成、信号处理和雷达任务时序调度层面，对阵元资源、CPU 资源和 GPU 资源的一体化管理研究较少。其次，通用集群调度平台虽然具备成熟的资源管理能力，但其资源模型主要面向标准化计算资源，难以直接表达数字阵系统中阵元的空间分布、阵面约束和复用关系。再次，部分调度研究重视算法效果，但对调度过程的工程化落地、日志记录、等待原因解释和 Web 可视化展示关注不足。

因此，本文在已有研究基础上，面向数字阵一体化资源管理场景，设计并实现一个基于 Web 的资源调度管理系统。系统将天线阵元、CPU 和 GPU 等异构资源纳入统一模型，在任务需求驱动下进行场景识别和联合调度，并通过调度日志、状态反馈和可视化界面展示调度过程。该研究既吸收了多资源调度和异构计算调度中的相关思想，也结合数字阵资源具有空间结构和任务约束复杂的特点，对通用调度方法进行了面向业务场景的系统化实现。

1.3 研究内容与创新点

围绕数字阵系统中多类异构资源统一管理和多任务联合调度问题，本文设计并实现了一套基于 Web 的数字阵一体化资源调度管理系统。系统以任务需求为驱动，将天线阵元、CPU 和 GPU 等资源纳入统一管理模型，在此基础上实现任务建模、资源状态维护、

场景识别、联合调度、日志记录和可视化展示等功能，形成从任务提交到资源分配、状态反馈和结果追踪的完整闭环。本文研究内容与技术路线如图 1-1 所示。

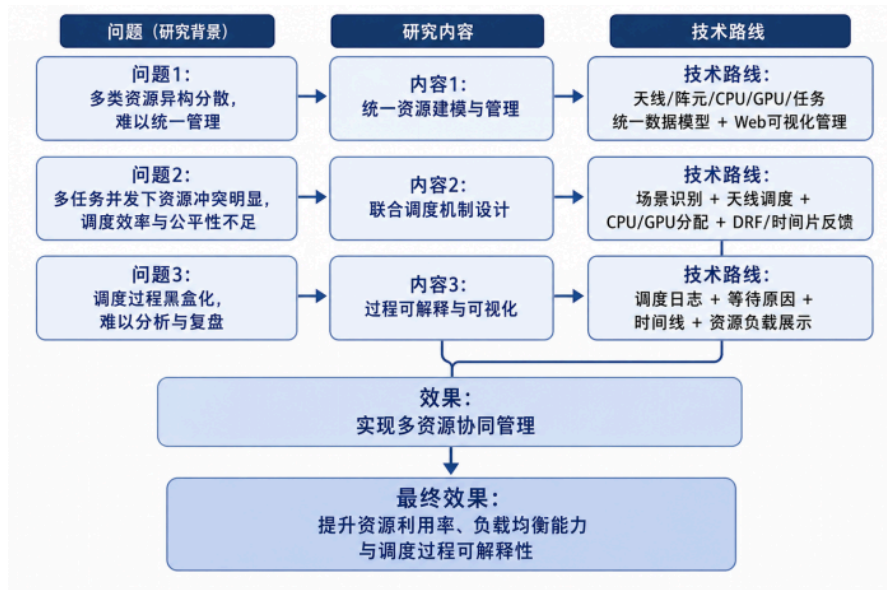


图 1-1 问题-内容-技术路线图

由图 1-1 可知，本文的研究工作主要围绕三个方面展开。

首先，针对多类资源异构分散、难以统一管理的问题，本文构建了面向数字阵任务的统一资源建模与管理机制。系统将天线、阵元、CPU、GPU 和任务对象纳入统一数据模型，通过资源表、任务表和分配记录表描述资源状态、任务需求与资源占用关系。同时，系统提供 Web 可视化管理界面，使用户能够对各类资源进行集中查看、状态维护和任务关联分析，为后续联合调度提供统一的数据基础。

其次，针对多任务并发下资源冲突明显、调度效率与公平性不足的问题，本文设计并实现了天线阵元与 CPU/GPU 计算资源的联合调度机制。系统根据任务的阵元需求、CPU 核数需求、GPU 显存需求、优先级、截止时间和依赖关系等信息进行场景识别，并在不同任务场景下选择合适的内部调度路径。天线侧支持单阵面优先和跨阵面扩展分配，计算侧支持紧凑式 CPU 分配、负载均衡式 CPU 分配和 GPU 单卡局部性分配，同时结合主导资源公平思想^[5]和时间片反馈机制，使系统能够在多资源约束条件下完成较为合理的协同分配。

最后，针对调度过程黑盒化、难以分析与复盘的问题，本文设计了调度过程可解释与可视化机制。系统在任务进入调度、等待、运行、时间片更新和完成释放等关键阶段记录调度日志，并保存场景识别结果、决策原因、等待原因和资源分配摘要。前端通过

时间线、任务状态和资源负载展示调度过程，使用户能够直观理解任务为何被调度、为何等待以及资源状态如何变化，从而提升系统运行过程的可观测性和可管理性。

在上述研究内容基础上，本文的创新点主要体现在以下三个方面。

（1）构建了面向数字阵任务的多类异构资源统一管理模型。不同于传统按设备或资源类型分散管理的方式，本文将天线阵元、CPU、GPU 和任务对象纳入同一调度框架，建立任务需求与资源供给之间的关联关系，为数字阵系统中的多资源协同调度提供了统一的数据基础。

（2）设计了面向任务场景的联合调度机制。系统不是对所有任务采用固定调度路径，而是根据阵元规模、计算资源需求、优先级、截止时间和依赖关系等特征识别任务场景，并在天线调度和计算资源分配之间进行协同决策。该机制能够兼顾低时延任务的快速响应、CPU 密集型任务的负载均衡、GPU 任务的局部性分配以及依赖任务的安全阻塞。

（3）形成了调度过程可解释的动态反馈机制。系统通过调度日志、等待原因记录、剩余时间写回、时间片反馈和前端时间线展示，对任务从提交、等待、运行到完成释放的过程进行持续记录。该机制增强了系统调度行为的可追踪性和可复盘性，使系统不仅能够给出资源分配结果，还能够说明任务为何这样调度、为何等待以及资源如何变化。

1.4 章节安排

本文围绕基于 Web 的数字阵一体化资源调度管理系统的设计与实现展开研究，全文共分为七章，各章内容安排如下。

第一章为绪论。主要介绍课题的研究背景与意义，梳理数字阵资源管理、多资源联合调度和 Web 可视化管理等相关研究现状，并说明本文的研究内容、技术路线与创新点。

第二章为系统总体设计。主要从系统需求出发，分析系统的功能需求与非功能需求，给出系统总体架构设计，并围绕资源、任务和调度过程建立数据库模型，为后续模块实现提供基础。

第三章为天线/CPU/GPU 联合调度设计。主要分析数字阵任务中的多资源约束关系，阐述天线阵元调度、CPU/GPU 计算资源分配以及三类资源协同分配流程，并说明调度主循环和任务状态更新机制。

第四章为任务场景识别与自动决策机制。主要介绍系统如何根据任务的阵元需求、计算资源需求、优先级、截止时间和依赖关系等信息识别任务场景，并根据不同场景选择相应的调度路径和决策目标。

第五章为调度日志与时间片反馈设计。主要介绍调度日志记录、等待原因保存、剩余时间写回、公平份额更新和时间片反馈机制，并说明前端时间线与资源负载展示对调度过程可解释性的支撑作用。

第六章为系统测试与结果分析。主要构建低时延任务、多资源均衡任务、GPU 辅助任务、CPU 密集型任务和依赖约束任务等典型测试场景，对系统的场景识别、资源分配、等待处理、负载均衡和接口安全性进行验证与分析。

第七章为结论与展望。主要总结本文完成的主要工作和系统实现效果，分析当前系统仍存在的不足，并对后续在调度策略优化、真实设备接入、测试体系完善和可视化增强等方面的工作进行展望。

第二章 系统总体设计

本章围绕数字阵一体化资源调度管理系统的总体设计展开说明。根据第一章对课题背景、研究现状和主要问题的分析，本系统需要同时解决多类异构资源统一管理、多任务资源协调分配以及调度过程可追踪等问题。因此，在进入具体调度算法和实现细节之前，有必要先从整体层面对系统需求、架构组成、业务流程和数据模型进行设计。

本章首先分析系统的功能需求与非功能需求，明确系统需要支持的资源管理、任务管理、调度控制、日志记录和可视化展示等能力；随后介绍系统总体架构，说明前端、后端、数据库以及核心模块之间的关系；接着梳理系统从任务提交到资源分配、状态反馈和结果展示的业务流程；最后对数据库设计思路和关键数据表进行说明，为后续天线阵元调度、CPU/GPU 联合调度、任务场景识别和调度反馈机制的实现提供基础。

2.1 系统需求分析

系统需求分析 数字阵一体化资源调度管理系统面向多任务并发条件下的异构资源管理场景，核心目标是在统一平台中完成天线阵元、CPU 和 GPU 等资源的集中维护、任务提交、自动调度和状态反馈。与普通设备管理系统不同，本系统不仅需要展示资源信息，还需要根据任务需求建立资源占用关系，并在调度过程中记录任务状态变化和资源分配结果。

从功能需求来看，系统首先需要具备资源统一管理能力。数字阵任务涉及的资源类型较多，其中阵元资源具有阵面分布和空间位置特征，CPU 资源具有核数和负载特征，GPU 资源具有显存容量和占用状态特征。系统应支持对这些资源进行统一建模、查询展示、状态维护和初始化管理，为后续调度提供准确的资源状态基础。

系统还需要具备任务管理能力。任务是资源调度的直接输入，系统应支持任务创建、任务查询、任务状态展示和历史记录查看等功能。任务信息需要包含任务名称、优先级、所需阵元数量、CPU 核数需求、GPU 显存需求、运行时长、截止时间、依赖关系和互斥关系等字段，从而较完整地描述任务对多类资源的需求及其运行约束。

其次，系统需要具备多资源联合调度能力。由于一个任务可能同时依赖阵元、CPU 和 GPU 三类资源，系统不能仅从单一资源角度判断任务是否可执行，而需要在任务提交后综合评估各类资源状态。对于满足条件的任务，系统应生成资源分配结果并更新任务

状态；对于暂时无法执行的任务，系统应保持其等待状态，并记录资源不足、依赖未满足或分配失败等原因，避免出现资源局部占用但任务整体无法运行的情况。

此外，系统需要具备调度过程记录与可视化展示能力。调度过程中产生的任务进入调度、等待、运行、时间片更新和完成释放等关键事件应被写入日志，便于后续查询、分析和复盘。同时，系统应通过 Web 页面展示资源负载、任务状态、资源分配结果和调度时间线，使用户能够直观了解当前系统运行情况和调度过程。

综上，本系统的功能需求可以概括为：以资源统一管理为基础，以任务需求建模为输入，以联合调度机制为核心，以日志记录和可视化反馈为支撑，实现数字阵多类资源在多任务场景下的协同管理与动态分配。

2.2 非功能需求分析

除资源管理、任务管理和联合调度等功能需求外，系统还需要满足可用性、安全性、可维护性、可扩展性和可解释性等非功能需求，以保证系统能够稳定支撑后续开发、测试和论文实验验证。

在可用性方面，系统采用 Web 方式提供操作入口，用户可以通过浏览器完成登录、资源查看、任务提交和调度结果查询等操作。前端页面应保持结构清晰、交互路径明确，并通过资源负载图、任务列表、状态标签和调度时间线等形式展示系统运行情况，使用户能够快速理解当前资源占用和任务执行状态。

在安全性方面，系统需要对关键操作进行访问控制。资源初始化、资源释放、任务提交和调度控制等操作会直接影响系统运行状态，如果缺少权限限制，可能导致资源状态被误修改或任务被异常提交。因此，系统应通过登录认证、令牌校验和接口鉴权等方式限制未授权访问，保证核心管理接口只能由已认证用户调用。

在可维护性方面，系统应保持清晰的模块边界。后端将认证鉴权、资源管理、任务管理、调度控制和公共能力拆分到不同模块中，避免业务逻辑相互耦合；前端则按照页面和组件组织代码，使资源展示、任务表单、状态监控和系统设置等功能相对独立。这样的结构有利于后续定位问题、修改功能和扩展模块。

在可扩展性方面，系统的数据模型和调度流程需要为后续变化预留空间。数字阵资源调度场景可能继续增加新的资源类型、任务约束或调度策略，因此任务模型应支持扩展新的约束字段，资源模型应能够容纳新的设备类型，调度模块也应便于接入新的场景识别规则和分配策略。

调度过程的可解释性同样是本系统的重要需求。资源调度系统不仅要给出最终分配结果，还应能够说明任务为何被执行、为何等待以及资源如何变化。为此，系统需要记录调度日志、等待原因、任务状态变化和资源分配摘要，并通过前端时间线和资源负载展示将调度过程可视化，为系统调试、实验分析和论文论证提供支撑。

2.3 系统总体架构

本系统采用前后端分离的总体架构，前端负责页面交互、资源展示、任务提交和调度过程可视化，后端负责用户认证、资源管理、任务管理、调度控制和数据持久化。前后端之间主要通过 REST API 进行数据交互，并通过 WebSocket 推送任务状态变化、资源告警和调度结果等实时信息。系统底层使用 MySQL 保存用户、资源、任务、分配记录和调度日志等数据，为调度执行和结果分析提供持久化支撑。

根据系统需求分析，本文采用前后端分离的总体架构设计。系统整体由用户层、前端表现层、通信层、后端服务层和数据存储层组成。用户通过浏览器访问系统前端页面，完成资源查看、任务提交和调度结果查看等操作；前端通过 REST API 与后端进行常规数据交互，并通过 WebSocket 接收任务状态变化和资源告警等实时消息；后端负责用户认证、资源管理、任务管理、联合调度、日志记录和数据持久化；数据库用于保存用户信息、资源数据、任务数据、分配记录和调度日志。系统总体架构如图 2-1 所示。

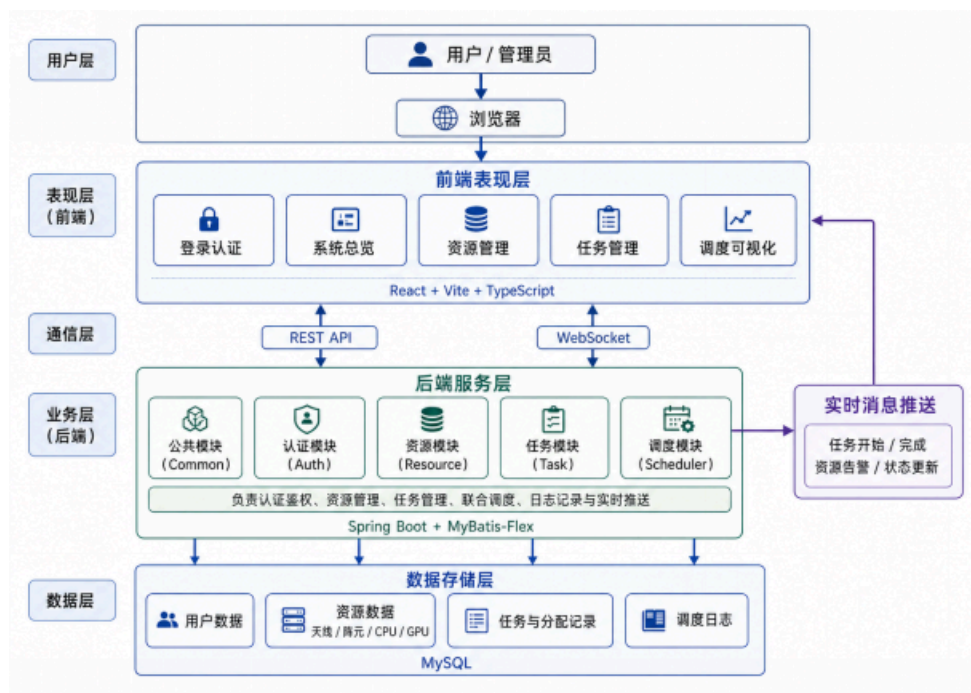


图 2-1 系统总体架构

从前端表现层来看，系统基于 React^[11]、Vite^[12] 和 TypeScript 实现，主要包含登录认证、系统总览、资源管理、任务管理和调度可视化等功能页面。登录认证页面用于完成用户身份校验，并为后续接口访问提供令牌；系统总览页面用于展示任务数量、资源负载和运行状态等概览信息；资源管理页面用于展示和维护天线阵元、CPU、GPU 等资源；任务管理页面支持任务创建、任务列表查看和任务状态跟踪；调度可视化页面则用于展示任务调度过程、资源分配结果和调度时间线。前端不仅是用户操作系统的入口，也是调度过程可视化和系统运行状态展示的重要载体。

后端服务层基于 Spring Boot^[13] 和 MyBatis-Flex 构建，并按照功能边界划分为多个模块。Common 模块提供统一返回结果、异常处理、JWT 工具、请求拦截和用户上下文等公共能力；Auth 模块负责用户登录、注册、令牌生成和认证校验；Resource 模块负责天线、阵元、CPU、GPU 及其分配记录的管理；Task 模块负责任务信息的保存、查询和状态维护；Scheduler 模块负责调度核心逻辑，包括任务场景识别、天线阵元调度、CPU/GPU 资源分配、时间片反馈、调度日志生成和 WebSocket 消息推送。通过这种模块化设计，系统能够将认证、资源、任务和调度逻辑相对分离，降低代码耦合度，并为后续扩展新的资源类型或调度策略提供便利。

通信层主要承担前后端数据交互和实时状态同步功能。普通业务请求采用 REST API 实现，例如用户登录、资源查询、任务创建和日志查询等；实时状态变化则通过 WebSocket 推送到前端，例如任务开始、任务完成、资源告警和任务状态更新等。REST API 保证了系统常规业务操作的稳定性，WebSocket 则提高了调度过程反馈的实时性，使用户无需频繁刷新页面即可观察任务和资源状态变化。

数据存储层使用 MySQL 作为持久化数据库，主要保存四类数据：用户数据、资源数据、任务与分配记录以及调度日志。用户数据用于支撑认证鉴权；资源数据用于描述天线、阵元、CPU 和 GPU 的基础属性及占用状态；任务与分配记录用于描述任务需求以及任务与具体资源之间的映射关系；调度日志用于记录任务调度过程中的关键事件和原因说明。通过这些数据表，系统能够将资源状态、任务状态和调度过程统一保存，为后续调度执行、前端展示和实验分析提供数据基础。

整体来看，该架构形成了“前端交互—后端调度—数据库持久化—实时反馈”的闭环。前端负责接收用户操作并展示系统状态，后端负责核心业务处理和调度决策，数据库保存系统运行过程中的关键数据，WebSocket 则将调度结果和状态变化及时反馈给前

端。该架构既满足了系统资源管理和任务调度的基本需求，也为后续联合调度机制、任务场景识别机制和调度日志反馈机制的实现奠定了基础。

2.4 系统业务流程

系统业务流程围绕任务从提交到完成的全过程展开，涵盖用户操作、后端调度以及数据存储之间的协同关系。整体流程以任务为主线，将资源状态查询、任务入库、调度决策、资源分配、状态反馈和资源释放等环节串联起来，形成闭环运行机制。系统业务流程如图 2-2 所示。

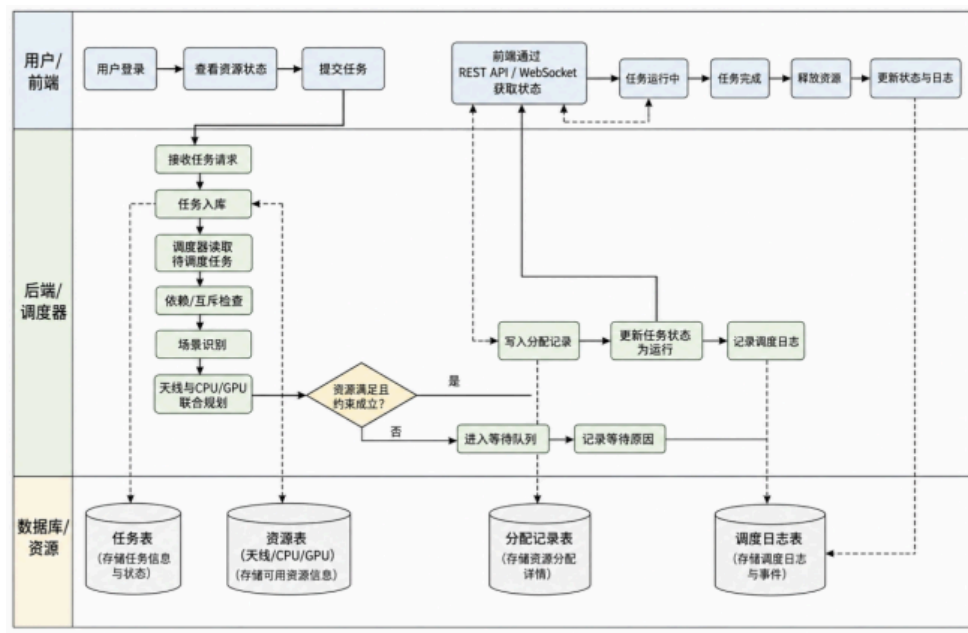


图 2-2 系统业务流程

从整体流程来看，系统首先由用户在前端发起操作。用户完成登录后，可以查看当前资源状态，包括天线阵元、CPU 和 GPU 的占用情况，并根据任务需求填写相关参数提交任务。前端将任务请求发送至后端，后端接收后将任务信息写入任务表，并将其标记为待调度状态。

在任务入库后，调度模块会周期性读取待调度任务，并结合当前资源状态和任务约束进行处理。系统首先对任务的依赖关系和互斥关系进行检查，若前置条件不满足，则任务进入等待状态，并记录具体原因；若约束条件满足，则进入场景识别阶段，根据任务的资源需求特征确定调度路径。

完成场景识别后，系统对天线阵元和 CPU/GPU 计算资源进行联合规划。在该过程中，调度模块需要查询资源表中的实时状态，判断各类资源是否能够同时满足任务需求。若资源满足且约束成立，则系统生成资源分配方案，并将分配结果写入分配记录表，同

时更新任务状态为运行中，并记录调度日志；若资源不足或分配失败，则任务进入等待队列，并将资源不足或冲突原因写入调度日志。

任务进入运行状态后，系统持续维护任务状态和资源占用情况。调度模块在每一轮调度循环中更新任务运行信息，并根据需要记录调度过程中的关键事件。前端通过 REST API 查询和 WebSocket 实时推送获取任务状态变化，包括任务开始、等待、运行和完成等信息，从而在页面中展示调度时间线和资源负载变化。

当任务运行完成后，系统释放其占用的天线阵元、CPU 和 GPU 资源，并更新相关资源状态和任务状态，同时记录任务完成日志。若任务未完成且仍需参与调度，则继续进入下一轮调度循环。通过上述流程，系统实现了从任务提交到资源释放的完整调度闭环。

2.5 数据库设计

为支撑系统中资源管理、任务调度与过程记录等功能，需要对多类数据进行统一建模与结构化存储。本系统采用关系型数据库 MySQL 进行数据持久化设计，并围绕任务驱动的资源调度过程，将系统数据划分为资源类数据、任务类数据和调度过程类数据三类，从而形成清晰的数据组织结构。系统数据库整体结构如图 2-3 所示。

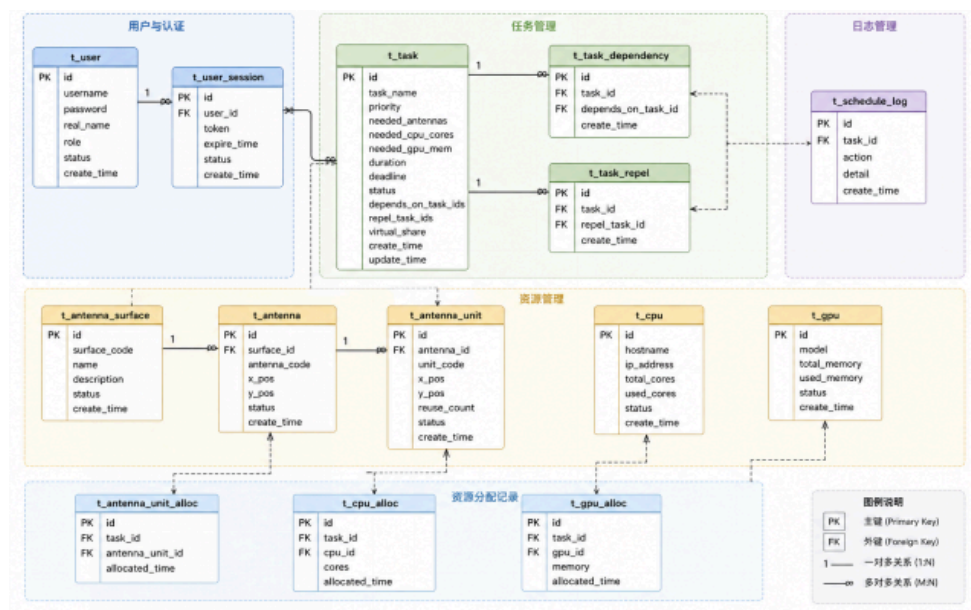


图 2-3 系统业务流程

数据库以任务表为核心，通过分配记录表将任务与具体资源建立关联，并通过调度日志表记录调度过程中的关键事件。资源表用于描述系统中可调度的各类资源状态，是调度决策的基础数据来源。

在资源数据方面，系统分别设计了天线阵面表、阵元表、CPU 资源表和 GPU 资源表，用于描述不同类型资源的属性及其当前状态。其中，阵元表用于刻画阵元在阵面中的分布及占用情况，CPU 表用于记录节点编号、核心数量及使用状态，GPU 表用于记录显存容量和占用情况。这些资源表共同构成系统可调度资源的基础集合，为调度模块提供实时资源视图。

在任务数据方面，系统通过任务表对任务进行统一描述。任务表中包含任务名称、优先级、阵元需求数量、CPU 核数需求、GPU 显存需求、运行时长、截止时间以及依赖关系等字段，用于完整表达任务在调度过程中的资源需求及约束条件。任务表既是调度模块的输入来源，也是前端展示任务状态的主要数据来源。

在资源分配方面，系统设计了多张分配记录表，用于记录任务与资源之间的映射关系。具体包括阵元分配表、CPU 分配表和 GPU 分配表，这些表分别记录任务所占用的具体阵元编号、CPU 核心编号以及 GPU 设备信息。通过分配记录表，可以实现资源占用状态的精确管理，同时为任务完成后的资源释放提供依据。

在调度过程记录方面，系统引入调度日志表，用于保存任务在调度过程中的关键事件信息。日志内容包括任务进入调度、资源不足等待、依赖条件不满足、资源分配成功、任务运行以及任务完成等状态变化，同时记录对应的原因说明和时间信息。通过该表，系统能够对调度过程进行完整追踪，为问题分析、实验验证和结果复盘提供数据支持。

此外，系统还设计了用户表和调度配置表。用户表用于实现系统登录认证和权限控制；调度配置表用于保存调度参数和策略配置，为后续调度策略调整提供支持。

总体而言，本系统的数据库设计以“任务驱动、资源支撑、过程记录”为核心思想，通过任务表、资源表、分配记录表和日志表之间的关联关系，实现了资源状态、任务状态和调度过程的统一管理。这一设计不仅满足系统功能实现的需求，也为后续联合调度算法、场景识别机制以及调度反馈机制的实现提供了可靠的数据基础。

2.6 引言

为进一步说明数据库结构对系统业务的支撑作用，本节对系统中的关键数据表进行归纳。系统数据表主要围绕任务、资源、分配记录和调度日志四类对象展开。其中，任务表用于描述任务需求和运行约束，资源表用于保存天线阵元、CPU 和 GPU 等可调度资源的状态，分配记录表用于建立任务与具体资源之间的占用关系，调度日志表用于记录调度过程中的关键事件。关键数据表设计说明如表 2-1 所示。

表 2-1 关键数据表设计说明

数据表名称	主要字段	功能说明	调度作用
t_task	id、 name、 priority、 needed_antennas、 needed_cpu_cores、 needed_gpu_mem、 duration、 remaining_seconds、 virtual_share、 status、 depends_on_task_ids、 repel_task_ids	保存任务基础信息、资源 需求和运行约束	作为调度输入的核心 数据来源
t_antenna	id、 code、 surface_code、 x_pos、 y_pos、 status	保存物理天线或阵面级 资源信息	支撑阵面管理与阵元 映射
t_antenna_unit	id、 antenna_id、 unit_code、 x_pos、 y_pos、 reuse_count、 surface_code、 status、 task_id	保存阵元级细粒度资源 信息	支撑阵元级调度与跨 阵面分配
t_cpu	id、 hostname、 ip_address、 total_cores、 used_cores、 status	保存 CPU 节点信息与当 前核数占用情况	支撑紧凑式和均衡式 CPU 分配
t_gpu	id、 model、 total_memory、 used_memory、 status	保存 GPU 资源信息及显 存占用情况	支撑单卡 GPU 局部性 分配
t_cpu_alloc	id、 task_id、 cpu_id、 cores	记录任务与 CPU 节点之 间的分配关系	支撑 CPU 占用记录与 资源释放
t_gpu_alloc	id、 task_id、 gpu_id、 memory	记录任务与 GPU 之间的 分配关系	支撑 GPU 占用记录与 资源释放
t_antenna_unit_alloc	id、 task_id、 antenna_unit_id	记录任务与阵元之间的 分配关系	支撑阵元映射、释放与 调试分析
t_schedule_log	id、 task_id、 action、 detail、 create_time	保存调度过程中的关键 动作和原因说明	支撑时间线展示、调度 解释和实验分析

从表中可以看出，t_task 是系统调度流程的核心入口。用户提交任务后，任务的资源需求、优先级、运行时长、依赖关系和互斥关系等信息都会写入该表，调度器随后根

据这些字段判断任务是否具备执行条件。因此，任务表不仅用于保存任务基本信息，也承担了调度输入模型的作用。

资源类数据表主要包括 `t_antenna`、`t_antenna_unit`、`t_cpu` 和 `t_gpu`。其中，`t_antenna` 和 `t_antenna_unit` 分别从物理天线和细粒度阵元两个层面对天线资源进行描述，使系统能够在阵面和阵元两个粒度上组织资源；`t_cpu` 和 `t_gpu` 分别保存计算节点和加速资源的当前状态，为 CPU 核数分配、GPU 显存判断和负载均衡提供数据依据。通过这些资源表，系统可以在调度前获取当前资源可用情况，并在任务运行过程中维护资源占用状态。

分配记录表主要包括 `t_cpu_alloc`、`t_gpu_alloc` 和 `t_antenna_unit_alloc`。这类表用于记录某个任务实际占用了哪些 CPU、GPU 和阵元资源，是连接任务表与资源表的中间结构。任务运行完成后，系统可以根据分配记录释放对应资源；在调试和实验分析阶段，也可以通过这些记录还原任务的实际资源分配结果。

`t_schedule_log` 表用于保存调度过程中的关键动作和原因说明，是系统实现可解释调度的重要数据基础。调度器在任务进入调度、资源不足等待、依赖阻塞、分配成功、任务运行和任务完成等阶段写入日志，使系统能够在前端以时间线形式展示任务状态变化。该表不仅服务于系统运行过程中的问题排查，也为后续实验结果分析提供了直接依据。

综上，系统关键数据表之间形成了较清晰的协作关系：任务表描述“要调度什么”，资源表描述“可以调度什么”，分配记录表描述“实际分配了什么”，调度日志表描述“为什么这样调度”。这种数据组织方式为系统实现资源管理、联合调度、状态反馈和过程追踪提供了基础支撑。

2.7 本章小结

本章围绕数字阵一体化资源调度管理系统的总体设计展开说明。首先，从功能需求和非功能需求两个方面明确了系统需要具备的资源管理、任务管理、联合调度、日志记录和可视化展示能力；随后，介绍了系统的前后端分离架构、后端模块划分以及前后端通信方式；在此基础上，结合系统业务流程图说明了任务从提交、入库、调度、运行到资源释放的完整闭环；最后，对数据库设计思想和关键数据表进行了分析，明确了任务表、资源表、分配记录表和调度日志表在系统中的作用。

通过本章设计，系统的整体结构、业务流程和数据支撑关系已经基本明确，为后续天线阵元、CPU 和 GPU 联合调度机制的设计与实现奠定了基础。

第三章 天线/CPU/GPU 联合调度设计

本章围绕数字阵任务中的天线阵元、CPU 和 GPU 联合调度问题展开设计。第二章已经从系统总体层面说明了需求、架构、业务流程和数据库设计，但系统能否真正完成多任务调度，关键在于后端调度器能否根据任务需求生成合理的资源分配方案。由于数字阵任务通常同时受到阵元数量、阵面分布、CPU 核数、GPU 显存、任务优先级和依赖关系等多种约束影响，因此需要将天线资源调度与计算资源调度放在统一流程中协同处理。

本章首先分析联合调度问题的特点，说明数字阵任务为什么不能按照单一资源进行独立分配；随后介绍天线阵元调度设计，重点说明频率冲突检测、复用次数约束以及 BFS、Dijkstra、贪心、堆优化和动态规划等阵元搜索策略；接着介绍 CPU/GPU 计算资源调度设计，说明紧凑式分配、负载均衡式分配和 GPU 单卡局部性分配等策略；最后给出联合调度流程与主循环设计，为后续任务场景识别与自动决策机制提供执行基础。

3.1 联合调度问题分析

数字阵任务在执行过程中通常不是单一资源驱动的任务，而是同时受到天线阵元资源、CPU 计算资源和 GPU 加速资源的共同约束。天线阵元决定任务能否在物理阵面上获得足够的信号收发支撑，CPU 资源决定任务是否具备基础计算能力，GPU 资源则主要用于部分需要加速处理的任务场景。因此，一个任务能否进入运行状态，并不取决于某一类资源是否单独满足，而取决于多类资源是否能够在同一调度周期内同时满足任务需求。

与通用计算任务相比，数字阵任务的资源约束具有更强的耦合性。对于普通计算集群而言，任务通常主要关注 CPU、内存或 GPU 等标准化资源，资源之间的空间结构关系相对较弱。而在数字阵系统中，阵元资源具有阵面分布、空间位置和复用状态等特征，不能简单视为数量相同的普通资源池。同时，任务在占用阵元资源后，还需要获得相应的 CPU 或 GPU 计算能力，才能完成后续处理过程。如果只完成阵元分配而计算资源不足，任务无法真正运行；如果只预留计算资源而阵元无法满足，也会造成计算资源空占。因此，天线阵元调度与计算资源调度必须放在统一流程中协同处理。

从系统运行角度看，联合调度需要解决的核心问题是任务需求、资源状态和分配结果之间的一致性。任务提交后，系统首先需要读取任务所需阵元数量、CPU 核数、GPU 显存、优先级、截止时间、依赖关系和互斥关系等信息；随后结合当前阵元、CPU 和 GPU 的可用状态判断任务是否具备运行条件。只有当任务的前置约束满足，且三类资源均能

形成可行分配方案时，系统才应写入分配记录并将任务置为运行状态。若其中任一条件不满足，任务应保持等待状态，并记录明确的等待原因。

联合调度还需要避免资源局部占用导致的整体失败问题。若系统在调度过程中先占用某一类资源，再继续尝试申请其他资源，则可能出现部分资源已经被锁定，但任务最终无法运行的情况。这不仅会降低资源利用率，还会影响后续任务的调度。因此，本系统采用整体可行性判断与统一落库的思路：调度器先分别规划天线阵元、CPU 和 GPU 资源，只有在三类资源方案均可行时，才统一写入分配记录并更新资源状态；如果任一资源规划失败，则不进行局部占用，而是记录失败原因并等待下一轮调度。

此外，多任务并发场景进一步增加了联合调度的复杂度。不同任务在阵元规模、计算需求、优先级、时延要求和依赖关系上存在差异，高优先级任务可能需要更快获得资源，CPU 密集型任务需要避免集中压在单个节点上，GPU 辅助任务则需要优先保证显存局部性。调度器不仅要判断资源是否足够，还要根据任务特征选择更合适的资源规划方式。由此可见，联合调度并不是简单地依次检查三类资源是否可用，而是需要在任务约束、资源状态和调度目标之间进行综合权衡。

综上，本文将数字阵任务调度抽象为天线阵元、CPU 和 GPU 三类资源共同约束下的联合分配问题。该问题的关键在于：在多任务并发条件下，保证任务只有在多类资源同时满足时才进入运行状态，并通过统一的分配记录和调度日志维护任务状态与资源状态的一致性。基于这一问题分析，后续小节将分别介绍天线阵元多策略搜索算法、CPU/GPU 计算资源分配策略以及三类资源协同工作的联合调度主循环。

3.2 天线阵元调度设计

天线阵元是数字阵系统中最具有业务特征的资源类型。与 CPU 核数、GPU 显存等数量型资源不同，阵元资源不仅需要关注可用数量，还需要考虑阵面归属、空间邻接关系、复用次数以及频率冲突等约束，这与相控阵天线系统中的阵面结构和阵元组织方式密切相关^[14]。因此，天线阵元调度不能简单理解为从资源池中取出若干空闲阵元，而应根据任务需求和阵元分布情况生成满足约束的阵元组合。

在本系统中，阵元调度的基本目标是在满足任务所需阵元数量的前提下，尽量降低资源冲突、提高分配质量，并在不同任务场景下选择合适的搜索策略。对于低时延任务，系统更关注快速得到可行解；对于高优先级小规模任务，系统更关注低复用、低冲突的高质量组合；对于大规模阵元任务，系统则需要提高跨阵面搜索能力，以增强获得可行解的概率。

3.2.1 阵元候选过滤与冲突检测

在执行具体搜索算法之前，系统首先对阵元候选集进行过滤。候选阵元需要满足两个基本条件：一是阵元当前复用次数没有超过设定阈值，二是阵元与当前任务波束之间不存在频率冲突。频率冲突检测的核心思想是遍历某个阵元上正在运行的波束，判断当前待分配波束与已有波束之间的频率间隔是否过近。如果频率间隔小于系统设定阈值，且两者不属于同一任务来源，则认为存在冲突，该阵元不能作为本轮任务的候选资源。

通过候选过滤，系统可以在搜索算法执行之前排除不可用阵元，减少无效搜索范围。对于剩余可用阵元，系统进一步根据复用次数、阵面归属和邻接关系进行排序或扩展搜索。复用次数较低的阵元通常具有更高的分配优先级，因为它们能够降低资源冲突概率，也有助于为后续任务保留更好的调度空间。

3.2.2 多策略阵元搜索算法

为了适应不同任务规模和调度目标，系统在天线阵元调度中设计了多种搜索策略，其中 BFS、Dijkstra 和动态规划等方法均属于经典图搜索与优化方法^[15-18]。不同策略的侧重点不同：有的强调搜索速度，有的强调路径权重，有的强调跨阵面扩展，有的适合小规模高质量搜索。系统可以根据任务场景、所需阵元数量、候选阵面数量和任务优先级等因素选择合适的阵元搜索方式。

第一类是基于 BFS 的快速可行搜索。该方法从某个起始阵元出发，沿着阵元邻接关系逐层扩展，直到找到满足任务所需数量的阵元组合。BFS 搜索过程较为直接，能够较快得到一个可行解，适合低时延任务或阵元需求较小的任务。其优点是实现简单、搜索速度较快，缺点是对复用次数和整体路径权重的优化能力有限。

第二类是基于 Dijkstra 的低权重路径搜索。该方法将阵元复用次数作为路径权重的一部分，在同一阵面内寻找累计权重较低的阵元组合。相比 BFS，Dijkstra 搜索更关注路径质量，能够优先选择复用次数较低的阵元，有利于减少资源冲突。但由于需要维护优先队列和路径权重，其搜索开销相对更大，适合对分配质量要求较高的任务。

第三类是基于贪心的邻域扩展搜索。该方法首先选择复用次数最低的可用阵元作为起点，然后在其邻接候选中持续选择复用次数较低、目标路径长度较合适的阵元加入路径。贪心策略的优势是计算过程较轻量，能够在较短时间内得到质量相对较好的组合；不足是容易受到局部最优影响，不能保证获得全局最优结果。因此，该方法更适合中等规模任务或对实时性有一定要求的场景。

第四类是基于堆结构的优先队列搜索。该方法将候选阵元放入优先队列中，并综合考虑阵元复用次数和阵面目标需求进行排序。与普通贪心搜索相比，堆优化搜索能够在更大的候选范围内动态选择当前代价较低的阵元，并可以支持跨阵面扩展。当单阵面不足以满足任务需求时，该策略可以从多个阵面中寻找可用阵元组合，提高大规模任务获得可行解的概率。

第五类是基于动态规划的最小权重路径搜索。该方法将路径长度和阵元权重作为状态变量，通过动态规划记录到达不同阵元、不同路径长度时的最小权重和，并在满足任务所需阵元数量时选择权重最小的路径。动态规划方法能够更系统地比较不同候选路径，适合高优先级、小规模且对分配质量要求较高的任务。但由于状态空间较大，该方法不适合所有任务统一使用，更适合作为特定场景下的高质量搜索策略。

上述多种算法的特点如表 3-1 所示。

表 3-1 天线阵元调度算法说明表

算法名称	核心思想	适用场景	调度特点
BFS 快速搜索	沿阵元邻接关系逐层扩展，优先获得可行路径	低时延任务、小规模阵元任务	速度较快，实现简单，但分配质量优化有限
Dijkstra 搜索	以复用次数作为权重，寻找低权重阵元路径	高优先级或对分配质量要求较高的任务	能够降低复用冲突，但搜索开销较大
贪心搜索	从低复用起点出发，持续选择邻域中较优阵元	中等规模任务、需要较快得到较优解的任务	计算开销较低，但可能陷入局部最优
堆优化搜索	通过优先队列动态选择低代价候选阵元	跨阵面任务、大规模阵元任务	支持较大范围搜索，提高可行解获得概率
动态规划搜索	记录不同路径长度下的最小权重组合	小规模高质量任务	分配质量较高，但状态空间和计算开销较大

3.2.3 调度策略选择机制

由于不同算法适用的任务场景不同，系统并不会对所有任务固定使用同一种阵元搜索方法，而是根据任务特征进行策略选择。低时延任务优先采用 BFS 等快速可行搜索方式，以减少调度时间；高优先级小规模任务更适合采用 Dijkstra 或动态规划等方式，以获得低复用、低冲突的阵元组合；当任务阵元需求较大或单阵面无法满足时，系统则倾向于采用堆优化搜索或跨阵面扩展策略，以提高可行解获得概率。

从整体流程看，天线阵元调度可以概括为以下步骤：首先根据当前任务需求和资源状态筛选可用阵元；然后按照阵面组织候选集合，并优先判断单阵面是否能够满足任务

需求；若单阵面可满足，则根据任务场景选择 BFS、Dijkstra、贪心或动态规划等搜索策略；若单阵面不可满足，则进入跨阵面扩展搜索；最后，如果生成满足条件的阵元集合，则将其作为候选方案返回给联合调度流程。

需要注意的是，天线阵元调度成功后，系统不会立即将阵元状态写为占用，而是先将结果作为候选方案保留。只有当后续 CPU/GPU 资源规划也成功时，系统才会统一写入分配记录并更新资源状态。若计算资源规划失败，则本轮阵元候选方案不会落库，任务继续保持等待状态。这样的设计能够避免阵元资源被局部占用，保证任务状态、资源状态和分配记录之间的一致性。

综上，天线阵元调度设计的核心在于将阵元资源从静态设备信息转化为可过滤、可搜索、可组合和可释放的调度资源。系统通过频率冲突检测、低复用优先、多策略搜索和跨阵面扩展等机制，使阵元资源能够更好地参与多资源联合调度，为后续 CPU/GPU 资源规划和联合调度主循环提供基础。

3.3 CPU/GPU 计算资源调度设计

CPU 和 GPU 是数字阵任务执行过程中的主要计算资源。与天线阵元资源相比，CPU 和 GPU 不具有阵面空间结构，但它们直接影响任务能否获得足够的计算能力。CPU 资源主要用于支撑任务的基础计算过程，GPU 资源则用于部分需要加速处理的任务场景。因此，在联合调度流程中，计算资源调度需要根据任务的 CPU 核数需求、GPU 显存需求、当前节点负载以及任务依赖关系生成可行的计算资源分配方案。

在算法原型设计阶段，计算资源调度被抽象为元任务到服务器节点的部署问题。对于一个待部署的元任务，系统需要判断某个服务器是否具有足够的空闲 CPU 核心和 GPU 资源，同时还需要检查该任务与服务器上已有任务之间的依赖和互斥关系。只有当计算资源容量满足要求，且依赖、互斥约束不冲突时，任务才能被部署到对应服务器上。该思想在原型代码中体现为 `can_assign`、`depends_and_repels` 和 `individual_deploy` 等逻辑，用于判断任务是否能够被安全分配到某个服务器节点上。

3.3.1 CPU 资源分配策略

CPU 调度的核心问题是如何在满足任务核心数需求的同时，兼顾任务启动效率和节点负载均衡，这与操作系统和集群系统中的任务调度、资源分配问题具有一定联系^[19,20]。对于普通任务而言，CPU 需求通常较小，系统更关注能否尽快为其找到可用计算节点；对于 CPU 密集型任务而言，任务会占用较多核心，如果仍将大量核心集中分配到单个节

点，容易造成局部节点负载过高，影响后续任务调度。因此，本系统在 CPU 侧采用紧凑式分配和负载均衡式分配两类思路。

紧凑式分配适用于 CPU 需求较低的普通任务。该策略优先从剩余核心数较多的节点中分配资源，尽量用较少节点满足任务需求。这样能够减少任务资源映射的复杂度，也便于任务完成后的资源释放。其缺点是当任务规模较大时，容易造成单个节点负载偏高，因此不适合作为 CPU 密集型任务的主要分配方式。

负载均衡式分配适用于 CPU 核数需求较高的任务。该策略不追求将任务压缩到尽可能少的节点上，而是根据各节点当前已使用核心数和剩余核心数，将任务所需核心分散到多个负载较低的节点中。为了评价分配结果的均衡程度，系统可以计算各节点剩余计算资源或负载比例的方差。方差越小，说明节点之间负载差异越小，分配结果越均衡。原型代码中的 `fitness_function` 就体现了这一思想：通过统计服务器 CPU 与 GPU 的剩余资源方差，对候选部署方案进行评价，从而选择负载更加均衡的资源分配结果。

从实现角度看，CPU 负载均衡分配可以概括为以下过程：首先筛选处于可用状态且仍有空闲核心的 CPU 节点；随后根据节点当前负载对节点进行排序；在分配过程中，优先选择当前负载较低的节点，并在每次分配后更新节点负载状态；当任务所需核心数全部满足时，生成 CPU 分配方案。若所有节点剩余核心数之和仍无法满足任务需求，则计算资源规划失败，任务进入等待状态。

3.3.2 GPU 资源分配策略

GPU 调度主要围绕显存容量和资源局部性展开。与 CPU 核心可以拆分到多个节点不同，本系统中的 GPU 资源采用单卡局部性分配原则。也就是说，当任务存在 GPU 显存需求时，系统优先寻找一张剩余显存能够满足任务需求的 GPU，而不是将一个任务的显存需求拆分到多张 GPU 上。

采用单卡局部性分配有两个原因。第一，GPU 资源通常具有较强的设备局部性，如果将一个任务拆分到多张 GPU 上，会增加跨卡协调和状态维护复杂度。第二，当前系统主要面向资源调度管理与验证场景，GPU 辅助任务规模相对有限，单卡满足能够降低实现复杂度，并使资源分配和释放过程更加清晰。因此，系统在 GPU 调度时会先筛选可用 GPU，再计算每张 GPU 的剩余显存，并优先选择能够独立满足任务需求的 GPU 作为候选资源。

若任务不需要 GPU，则系统不会为其分配 GPU 资源；若任务需要 GPU 但不存在单张 GPU 能够满足显存需求，则计算资源规划失败，任务进入等待状态，并在调度日志中记录 GPU 资源不足或计算资源规划失败的原因。

3.3.3 计算资源原型算法与系统落地

在原型设计中，计算资源部署还尝试使用遗传算法进行优化。遗传算法是一类经典启发式优化方法，可用于复杂搜索空间中的候选方案优化^[21]。其基本思想是：首先随机生成若干可行部署个体，每个个体表示一组元任务到服务器节点的映射关系；随后通过适应度函数评价部署结果的负载均衡程度；再通过选择、交叉和变异等操作迭代生成新的部署方案，最终选择适应度更优的个体。原型代码中的 `create_individual`、`fitness_function` 和 `genetic_algorithm` 分别对应初始个体生成、负载均衡评价和遗传迭代过程。

不过，从当前系统实现和论文工程目标出发，直接将遗传算法作为在线调度核心并不一定合适。一方面，遗传算法需要维护种群并进行多轮迭代，在线调度耗时和稳定性不易控制；另一方面，本系统更强调调度过程的可解释性，需要用户能够理解任务为何被分配到某些 CPU 或 GPU 资源。因此，当前系统实际落地时更适合采用规则明确、结果可解释的启发式分配方式，即以紧凑式分配、负载均衡式分配和 GPU 单卡局部性分配作为主要计算资源规划策略，并借助负载方差评价思想衡量分配结果的均衡程度。

这种处理方式保留了算法原型中“负载均衡优化”的核心思想，同时降低了工程实现复杂度，使系统更适合作为毕业设计中的资源调度管理平台。若后续需要进一步提升全局优化能力，可以在当前启发式调度框架基础上继续引入遗传算法、模拟退火或其他智能优化方法作为离线优化或候选方案生成模块。

3.3.4 计算资源调度结果处理

与天线阵元调度类似，CPU/GPU 资源规划成功后，系统不会立即单独写入资源占用状态，而是将计算资源结果作为候选方案返回给联合调度流程。只有当天线阵元规划和 CPU/GPU 资源规划都成功时，系统才会统一写入资源分配记录，并将任务状态更新为运行中。如果计算资源规划失败，则不会占用任何 CPU 或 GPU 资源，而是记录等待原因并使任务继续保持等待状态。

综上，CPU/GPU 计算资源调度设计的核心在于根据任务计算需求和资源负载状态生成可解释的资源分配方案。CPU 侧通过紧凑式分配和负载均衡式分配兼顾启动效率与

节点负载，GPU 侧通过单卡局部性分配降低跨卡调度复杂度。同时，原型中的负载方差评价和遗传算法思想为后续进一步优化计算资源部署提供了扩展方向。

3.4 联合调度流程与主循环

在前两节中，本文分别介绍了天线阵元调度算法和 CPU/GPU 计算资源调度策略。为了使这些算法能够在系统中协同工作，还需要通过统一的联合调度流程将任务队列、资源状态、场景识别、资源规划和状态更新组织起来。联合调度流程的核心目标是保证任务只有在阵元、CPU 和 GPU 三类资源均满足需求时才进入运行状态，并避免出现部分资源已被占用但任务整体无法执行的情况。

联合调度流程如图 3-1 所示。

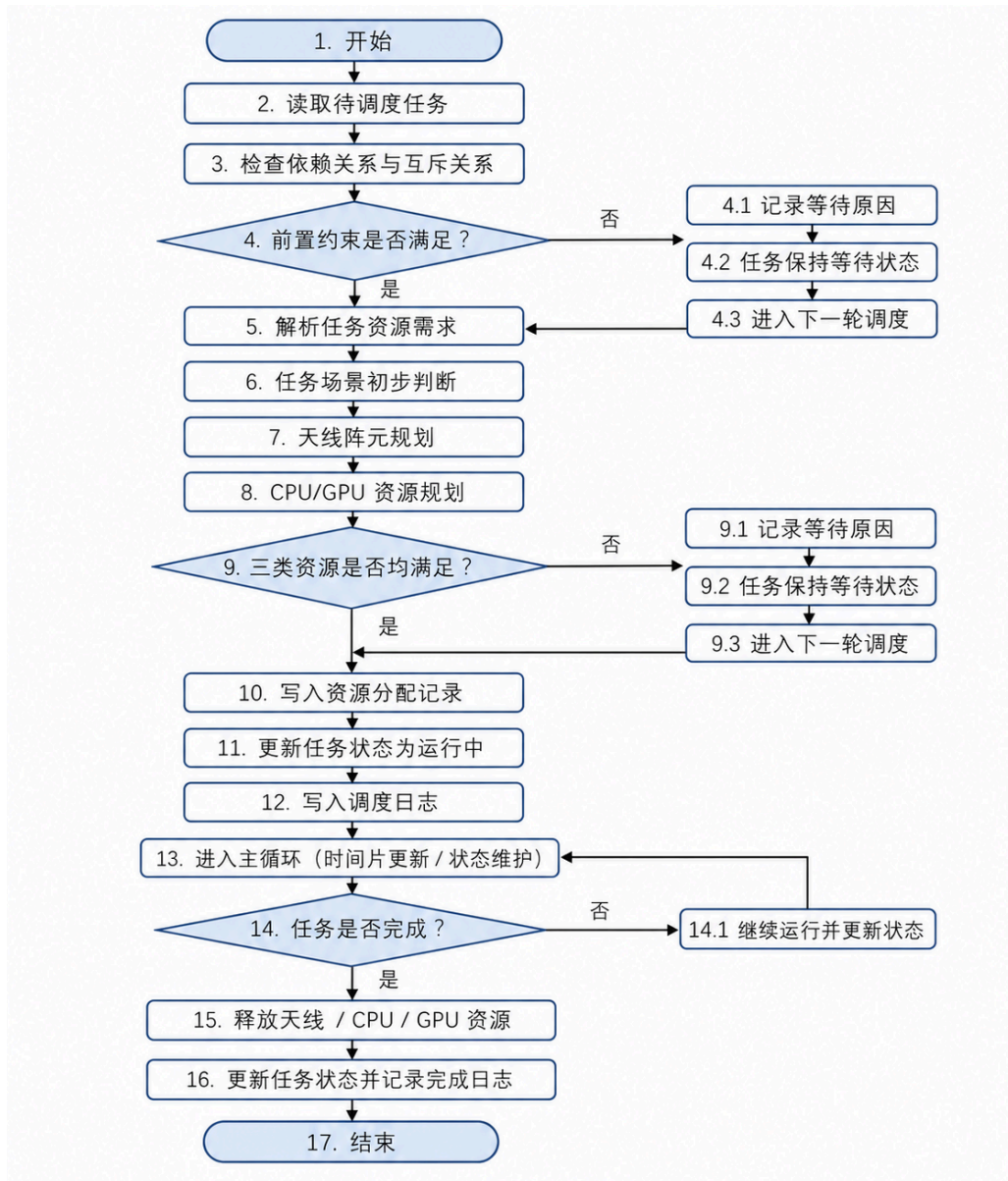


图 3-1 联合调度流程图

由图 3-1 可知，系统调度流程从读取待调度任务开始。调度器周期性扫描任务队列，获取处于待调度或等待状态的任务，并读取任务的阵元需求、CPU 核数需求、GPU 显存需求、优先级、截止时间、运行时长、依赖关系和互斥关系等信息。这些信息是后续场景识别和资源规划的输入。

在进入资源规划之前，系统首先进行依赖关系和互斥关系检查。若任务依赖的前置任务尚未完成，或该任务与当前运行任务存在互斥关系，则系统不会继续执行天线阵元和 CPU/GPU 资源搜索，而是直接将任务保持在等待状态，并写入对应等待原因。这样可以避免任务在前置条件不满足时错误占用资源，也减少不必要的搜索开销。

当前置约束满足后，系统会根据任务特征进行场景识别。场景识别主要依据任务所需阵元数量、CPU 核数需求、GPU 显存需求、优先级、截止时间和资源状态等因素进行判断。识别结果并不直接等同于具体算法名称，而是用于确定本轮调度目标。例如，低时延任务更强调快速获得可行解，高优先级小规模任务更强调低复用和低冲突，CPU 密集型任务更强调计算节点负载均衡，GPU 辅助任务则更强调单卡局部性。

完成场景识别后，调度器首先调用天线阵元调度模块生成阵元候选方案。系统会根据任务场景选择 BFS、Dijkstra、贪心、堆优化或动态规划等搜索策略，并结合频率冲突检测、复用次数和阵面分布等条件生成可用阵元集合。若阵元规划失败，系统会记录阵元资源不足或阵元组合失败等等待原因，并终止该任务本轮调度。

若阵元候选方案生成成功，调度器继续执行 CPU/GPU 计算资源规划。CPU 侧根据任务特征选择紧凑式分配或负载均衡式分配，GPU 侧则根据单卡局部性原则寻找能够满足显存需求的 GPU。若计算资源规划失败，系统同样不会占用任何资源，而是记录计算资源不足或 GPU 显存不足等原因，并等待下一轮调度重新评估。

当阵元、CPU 和 GPU 三类资源候选方案均可行时，系统才会统一写入分配记录，并将任务状态更新为运行中。具体而言，系统会写入阵元分配记录、CPU 分配记录和 GPU 分配记录，同时更新对应资源的占用状态。由于资源状态更新在三类资源均可行之后统一执行，因此可以避免“阵元已占用但 CPU/GPU 不足”或“计算资源已占用但阵元不可用”等不一致情况。

除资源可行性判断外，调度主循环还需要处理多任务之间的排序问题。对于多个待调度任务，系统不能简单按照提交顺序依次处理，而需要综合考虑任务优先级、主导资源份额和虚拟公平份额等因素。主导资源份额用于衡量任务在各类资源中占用比例最高的资源维度，该思想借鉴了多资源公平调度中的 DRF 方法^[5]；虚拟公平份额用于反映任

务在历史调度过程中的累计资源使用情况。通过这类排序机制，系统能够在多任务并发条件下兼顾高优先级响应和资源使用公平性。

从运行机制上看，联合调度并不是一次性静态分配，而是一个周期性执行的主循环。调度器在每轮循环中处理两类任务：一类是待调度或等待中的任务，系统会重新检查其约束条件和资源可用情况；另一类是运行中的任务，系统会根据时间片推进更新其剩余执行时间和公平份额。当任务运行完成后，系统释放其占用的阵元、CPU 和 GPU 资源，并写入完成日志。资源释放后，等待队列中的任务会在后续调度周期中重新获得评估机会。

联合调度主循环可概括为 算法 3-1。

算法 3-1 天线/CPU/GPU 联合调度主循环

输入: Q , 任务队列 U , 阵元资源集合 C , CPU 资源集合 G , GPU 资源集合**输出:** 任务状态、资源状态、分配记录和调度日志

// 处理运行中任务

for task in running_tasks:

更新 task.remaining_seconds 和 task.virtual_share

if task.remaining_seconds <= 0:

释放 task 占用的阵元、CPU 和 GPU 资源

更新 task.status = “COMPLETED”, 并写入完成日志

end

end

// 处理待调度任务

根据优先级、主导资源份额和虚拟公平份额对 pending_tasks 排序

for task in pending_tasks:

if 依赖关系或互斥关系不满足:

更新 task.status = “WAITING”, 并写入等待日志

continue

end

根据任务需求、优先级和截止时间识别任务场景

antenna_plan = AntennaSchedule(task, U)

compute_plan = ComputeSchedule(task, C, G)

if antenna_plan = none or compute_plan = none:

更新 task.status = “WAITING”, 并写入资源等待原因

continue

end

统一写入阵元、CPU 和 GPU 分配记录

更新资源占用状态和 task.status = “RUNNING”

写入调度成功日志, 并向前端推送状态变化

end

进入下一轮调度周期

算法 3-1 体现了系统联合调度的两个关键设计思想。第一, 系统采用“先规划、后落库”的方式, 调度器不会在单类资源规划成功后立即占用资源, 而是等待三类资源方案全

部可行后统一写入分配结果。第二，系统采用周期性主循环推进任务状态，使等待任务、运行任务和完成任务都能够在同一调度框架下持续更新。

通过上述流程，系统形成了“任务排序—约束检查—场景识别—天线规划—计算资源规划—统一落库—状态反馈”的完整闭环。该闭环将天线阵元算法、CPU/GPU 分配策略和任务状态管理结合起来，使系统能够在多任务并发条件下完成资源协同分配，并保证任务状态、资源状态和调度日志之间的一致性。

在正式执行资源规划之前，系统首先进行依赖关系和互斥关系检查。若任务依赖的前置任务尚未完成，或当前存在与该任务互斥的运行任务，则说明该任务暂不具备启动条件。此时系统不会继续执行天线和计算资源搜索，而是直接记录等待原因，并保持任务等待状态，等待下一轮调度重新评估。将约束检查放在资源规划之前，可以减少无效资源搜索，也能避免任务在前置条件不满足时错误占用资源。

当前置约束满足后，系统进一步解析任务资源需求，并进行任务场景的初步判断。场景判断主要依据任务的阵元需求规模、CPU 核数需求、GPU 显存需求、优先级和截止时间等信息。该步骤并不直接完成资源分配，而是为后续选择不同资源规划路径提供依据。例如，低时延任务更强调快速获得可行解，CPU 密集型任务更强调计算节点之间的负载均衡，GPU 辅助任务则需要优先保证单卡显存可行性。

完成任务需求解析后，系统依次进行天线阵元规划和 CPU/GPU 资源规划。天线阵元规划主要判断当前阵元资源是否能够满足任务所需数量，并根据任务场景选择单阵面优先或跨阵面扩展等分配方式；CPU/GPU 资源规划则根据任务计算需求判断 CPU 核数和 GPU 显存是否满足要求，并生成对应的计算资源候选方案。需要注意的是，在这一阶段生成的方案只是候选结果，系统不会立即写入资源占用状态。

随后，系统对三类资源规划结果进行统一可行性判断。只有当天线阵元、CPU 和 GPU 均满足任务需求时，系统才会写入资源分配记录，并将任务状态更新为运行中，同时生成调度日志。若其中任一类资源无法满足，系统不会保留局部分配结果，而是记录资源不足或规划失败的原因，将任务继续保持在等待状态，并进入下一轮调度。这样的设计保证了任务状态、资源状态和分配记录之间的一致性。

任务进入运行状态后，调度流程并不会立即结束，而是进入调度主循环中的运行维护阶段。在该阶段，系统会按照时间片周期更新任务剩余运行时间、资源占用状态和调度日志。若任务尚未完成，则继续运行并在后续周期中维护状态；若任务达到完成条件，

则释放其占用的天线阵元、CPU 和 GPU 资源，更新任务状态为完成，并记录完成日志。资源释放后，调度器可以在后续循环中重新评估等待队列中的其他任务。

从整体上看，本系统的联合调度主循环具有两个特点。一方面，系统采用“先规划、后落库”的方式，只有在三类资源均可行时才统一写入分配记录，避免了部分资源占用造成的整体调度失败；另一方面，系统通过周期性调度循环持续维护任务状态和资源状态，使任务能够在等待、运行和完成之间动态流转。这种设计使系统具备较完整的多资源联合调度闭环，也为后续任务场景识别、调度日志记录和动态反馈机制提供了执行基础。

3.5 本章小结

本章围绕天线阵元、CPU 和 GPU 三类资源的联合调度设计展开说明。首先分析了数字阵任务在执行过程中面临的多资源耦合约束，指出任务能否运行不仅取决于单类资源是否可用，还取决于阵元、CPU 和 GPU 是否能够在同一调度周期内形成一致的分配结果。在此基础上，本章分别介绍了天线阵元调度和 CPU/GPU 计算资源调度的设计思路。

在天线阵元调度方面，系统结合阵元复用次数、频率冲突、阵面分布和邻接关系等因素，设计了 BFS 快速搜索、Dijkstra 低权重路径搜索、贪心搜索、堆优化搜索和动态规划搜索等多种阵元搜索策略，使系统能够根据不同任务场景在搜索速度、分配质量和跨阵面可行性之间进行权衡。在计算资源调度方面，系统根据 CPU 核数需求和 GPU 显存需求，采用紧凑式 CPU 分配、负载均衡式 CPU 分配和 GPU 单卡局部性分配等策略，从而兼顾任务启动效率、计算负载均衡和资源释放维护的清晰性。

本章还进一步说明了联合调度流程与主循环。系统通过任务排序、依赖与互斥检查、场景识别、天线阵元规划、CPU/GPU 资源规划和统一落库等步骤，将多类资源调度策略组织为完整闭环。只有当阵元、CPU 和 GPU 三类资源方案均可行时，系统才会写入分配记录并更新任务状态；若任一环节失败，则记录等待原因并进入下一轮调度。该设计保证了任务状态、资源状态和分配记录之间的一致性，也避免了局部资源占用导致整体调度失败的问题。

通过本章设计，系统已经具备多资源联合调度的执行基础。下一章将在此基础上进一步介绍任务场景识别与自动决策机制，重点说明系统如何根据任务特征选择合适的调度目标和资源规划路径。

第四章 任务场景识别与自动决策机制

第三章已经说明了系统中天线阵元、CPU 和 GPU 三类资源的具体调度方法及联合调度主循环。但在实际使用过程中，用户通常只提交任务需求，而不会直接选择某一种内部调度算法。若系统要求用户手动指定 BFS、Dijkstra、动态规划、负载均衡分配等策略，不仅会增加使用复杂度，也难以体现资源调度系统的自动化能力。因此，在具体资源调度算法之上，还需要设计任务场景识别与自动决策机制。

本章围绕任务场景识别与自动决策机制展开说明。首先介绍该机制的总体设计思路，说明其在任务提交与联合资源规划之间承担的作用；随后从约束层、资源层和策略层三个角度分析场景识别规则，归纳低时延任务、多资源均衡任务、GPU 辅助任务、高优先级小规模任务、大规模阵元任务、CPU 密集型任务和依赖约束任务等典型场景；接着说明不同任务场景与调度目标、资源规划路径之间的映射关系；最后介绍决策结果如何写入调度日志并通过前端展示，从而提升调度过程的可解释性。

4.1 设计思路

第三章已经说明了系统中天线阵元、CPU 和 GPU 三类资源的具体调度方法，包括天线侧的多策略阵元搜索、计算侧的 CPU/GPU 资源规划以及联合调度主循环。然而，在实际使用过程中，用户提交任务时通常只描述任务需求，例如需要多少阵元、需要多少 CPU 核数、是否需要 GPU、任务优先级如何以及是否存在依赖关系，而不会主动指定系统内部应采用 BFS、Dijkstra、贪心搜索、动态规划或负载均衡分配等具体策略。如果要求用户手动选择调度算法，不仅会增加使用复杂度，也不符合资源调度系统自动化管理的目标。

因此，本文在联合调度流程基础上设计了任务场景识别与自动决策机制。该机制的核心思想是：由用户提交任务约束信息，系统根据任务特征和当前资源状态自动判断任务所属场景，再将场景识别结果转换为相应的调度目标，最后由联合调度模块选择合适的天线阵元搜索策略和 CPU/GPU 资源分配方式。通过这种方式，系统能够从“人工选择调度策略”转变为“系统自动识别任务场景并选择调度路径”。

任务场景识别的输入主要包括任务自身属性和系统资源状态两类信息。任务自身属性包括所需阵元数量、CPU 核数需求、GPU 显存需求、优先级、运行时长、截止时间、依赖关系和互斥关系等；系统资源状态则包括当前阵元可用情况、阵元复用次数、CPU

节点负载、GPU 显存占用以及正在运行任务的状态。系统综合这些信息后，可以判断任务更偏向低时延任务、多资源均衡任务、GPU 辅助任务、高优先级小规模任务、大规模阵元任务、CPU 密集型任务或依赖约束任务等类型。

自动决策机制的输出并不是简单的算法名称，而是更接近调度目标的解释性结果。例如，对于低时延任务，系统的目标是快速获得可行解，因此更倾向于选择搜索开销较低快速可行路径；对于高优先级小规模任务，系统更关注低复用和低冲突的阵元组合，因此更适合采用低权重路径搜索或高质量组合搜索；对于 CPU 密集型任务，系统更关注多个 CPU 节点之间的负载均衡；对于 GPU 辅助任务，系统则优先保证单卡显存可行性。这样的设计能够降低用户理解成本，也便于后续通过调度日志和前端界面解释系统行为。

从系统结构上看，任务场景识别与自动决策机制位于任务提交和联合资源规划之间。它既不直接替代第三章中的资源调度算法，也不是单独存在的规则模块，而是承担“任务特征分析”和“调度目标选择”的中间层作用。通过该机制，系统可以根据不同任务特征动态调整后续资源规划方向，使天线阵元调度、CPU/GPU 资源分配和调度日志输出能够围绕同一任务场景协同工作，从而提高联合调度流程的适应性和可解释性。

4.2 场景识别规则

任务场景识别的目标，是在任务进入具体资源规划之前，对任务的主要约束特征进行判断，从而为后续调度目标选择提供依据。由于数字阵任务同时涉及阵元、CPU 和 GPU 等多类资源，系统不能只依据单一字段进行分类，而需要综合任务约束、资源需求和当前资源状态进行判断。为保证识别过程清晰稳定，本文将场景识别规则划分为约束层、资源层和策略层三个层次。

约束层主要判断任务是否具备进入资源规划流程的前置条件。系统首先检查任务是否存在未完成的依赖任务，或是否与当前运行任务存在互斥关系。若依赖关系尚未满足，或互斥条件阻止任务启动，则系统不会继续执行阵元、CPU 和 GPU 资源规划，而是将任务保持在等待状态，并记录依赖阻塞或互斥冲突等原因。约束层识别的作用在于提前过滤暂不具备执行条件的任务，避免无效搜索和错误分配。

资源层主要判断任务更受哪一类资源约束影响。系统会读取任务所需阵元数量、CPU 核数需求和 GPU 显存需求，并结合当前阵元可用数量、阵元复用次数、CPU 节点负载和 GPU 显存占用情况进行分析。例如，当任务存在 GPU 显存需求时，系统优先将其识别为 GPU 辅助任务；当 CPU 核数需求明显较高时，系统倾向于将其识别为 CPU 密

集型任务；当阵元需求规模较大，或单阵面可用阵元难以满足任务需求时，系统会进一步识别其是否属于大规模阵元任务或跨阵面扩展任务。

策略层则在约束条件和资源特征已经明确的基础上，结合任务优先级、截止时间和调度目标进行进一步判断。对于截止时间较短、资源需求较小的任务，系统更关注快速获得可行解；对于优先级较高但任务规模较小的任务，系统更关注降低阵元复用和冲突；对于阵元与计算资源需求较均衡的任务，系统更关注整体资源利用率和调度公平性。通过这种分层识别方式，系统可以避免将场景识别退化为简单的单条件判断，使识别结果更加稳定，也更便于后续解释调度行为。

任务场景识别规则如表 4-1 所示。

表 4-1 任务场景识别规则说明表

场景类型	主要识别条件	典型任务特征	调度目标
低时延任务	截止时间较短，阵元和 CPU 需求较小	任务规模较小，强调快速响应	快速得到可行解
多资源均衡任务	阵元和 CPU 需求中等，无极端时限约束	多类资源均有需求，无明显单一瓶颈	兼顾解质量、利用率与公平性
GPU 辅助任务	存在 GPU 显存需求	需要少量 GPU 加速资源	优先单卡满足
高优先级小规模任务	优先级较高，阵元需求较小	规模不大，但要求较高分配质量	优先低复用、低冲突组合
大规模阵元任务	阵元需求较大，单阵面可能不足	需要跨阵面组合可用阵元	保证可行性与扩展能力
CPU 密集型任务	CPU 核数需求明显较高	容易造成单节点负载过高	提高 CPU 节点间负载均衡性
依赖约束任务	存在未完成依赖或互斥关系	当前不具备立即启动条件	暂缓调度并记录阻塞原因

由表 4-1 可知，不同任务场景之间的差异主要体现在资源瓶颈和调度目标上。低时延任务强调快速响应，因此更适合采用搜索开销较低的快速可行路径；高优先级小规模任务虽然规模不大，但对分配质量要求更高，因此更关注低复用、低冲突的阵元组合；大规模阵元任务的核心问题是单阵面可用资源可能不足，需要通过跨阵面扩展提高可行解获得概率；CPU 密集型任务则更关注计算节点之间的负载均衡；GPU 辅助任务主要关注是否存在单张 GPU 能够满足显存需求。

依赖约束任务与其他任务不同，其主要瓶颈并不是资源数量不足，而是前置条件尚未满足。因此，系统在识别到依赖或互斥约束不满足时，不会继续执行后续资源搜索，而

是直接记录等待原因并保留任务状态。这样既能减少无效调度，也能防止任务在不满足业务约束的情况下错误启动。

通过上述规则设计，系统能够在任务进入联合调度流程前完成初步分类，并为后续自动决策映射提供依据。该机制使调度器能够根据任务特征选择差异化的调度目标，而不是对所有任务采用统一的固定策略。

4.3 自动决策映射

完成任务场景识别后，系统需要将识别结果进一步转换为具体的调度目标和资源规划路径。这里的自动决策并不是简单地把某个场景固定绑定到某一个算法，而是根据任务场景确定本轮调度更应关注的目标，例如快速可行、低复用高质量、跨阵面扩展、计算负载均衡或依赖阻塞等待等。随后，联合调度模块再依据该目标选择相应的天线阵元搜索策略和 CPU/GPU 资源分配方式。

自动决策映射的作用在于连接“任务特征”和“具体调度算法”。对于用户而言，其只需要提交任务需求；对于系统而言，则需要根据任务特征自动判断调度方向。例如，低时延任务更强调快速响应，因此系统倾向于选择 BFS 等搜索开销较低快速可行路径；高优先级小规模任务更关注阵元复用次数和冲突情况，因此系统更倾向于选择 Dijkstra 或动态规划等低权重、高质量搜索方式；大规模阵元任务可能无法在单阵面内满足需求，因此系统更倾向于采用堆优化搜索或跨阵面扩展策略。计算资源侧也遵循类似思路：CPU 密集型任务优先采用负载均衡式分配，GPU 辅助任务则优先采用单卡局部性分配。

场景与决策之间的对应关系如表 4-2 所示。

表 4-2 场景与决策对应关系表

任务场景	典型特征	自动决策目标	资源规划路径
低时延任务	阵元需求小、CPU 需求低、时限紧	FAST_FEASIBLE 快速可行分配	优先采用 BFS 快速搜索，并采用紧凑式 CPU 分配
多资源均衡任务	阵元与 CPU 需求中等	FAIR_BALANCED 公平均衡分配	综合考虑阵元复用、CPU 负载和主导资源份额
GPU 辅助任务	存在 GPU 显存需求	GPU_LOCALITY_SINGLE_CARD 单卡局部性分配	天线侧生成可行阵元方案，GPU 侧优先单卡满足
高优先级小规模任务	优先级高、任务规模较小	QUALITY_LOW_REUSE 低复用高质量分配	优先采用 Dijkstra 或 DP 搜索低复用阵元组合
大规模阵元任务	阵元需求大、可能需要跨阵面	SCALABLE_CROSS_SURFACE 可扩展跨阵面分配	优先采用堆优化或贪心搜索进行跨阵面扩展
CPU 密集型任务	CPU 核数需求较高	COMPUTE_LOAD_BALANCE 计算负载均衡分配	CPU 侧采用负载均衡式分配，降低单节点热点
依赖约束任务	存在未满足依赖或互斥关系	DEPENDENCY_BLOCKED 依赖阻塞等待	不执行资源规划，直接记录等待原因

由表 4-2 可知，系统的自动决策并不是单纯输出场景名称，而是进一步确定后续资源规划的侧重点。低时延任务和高优先级小规模任务虽然都可能属于小规模任务，但二者的调度目标并不相同：前者更关注快速获得可行解，后者更关注低复用和低冲突的分配质量。因此，低时延任务更适合采用 BFS 快速搜索，而高优先级小规模任务则更适合采用 Dijkstra 或动态规划等低权重搜索方式。

对于计算资源相关任务，系统同样根据场景差异选择不同策略。GPU 辅助任务的核心约束是显存容量和设备局部性，因此系统优先寻找一张能够满足需求的 GPU；CPU 密集型任务的主要风险是大量核心集中到单个节点上，导致节点负载不均衡，因此系统优先采用负载均衡式 CPU 分配。多资源均衡任务则没有明显单一瓶颈，系统需要在阵元复用、CPU 负载和主导资源份额之间取得平衡。

依赖约束任务属于较特殊的一类。对于这类任务，系统首先判断前置条件是否满足。如果依赖任务尚未完成或存在互斥冲突，则系统不会进入天线阵元搜索和 CPU/GPU 资源规划阶段，而是直接将任务保持在等待状态，并写入依赖阻塞或互斥冲突原因。这样可以避免任务在业务约束不满足时错误启动，也能够减少无效资源搜索。

从实现逻辑看，自动决策映射机制位于场景识别和联合调度算法之间。场景识别负责判断任务属于哪一类，自动决策映射负责确定当前任务更应采用怎样的调度目标，最终再由第三章中的天线阵元搜索算法和 CPU/GPU 分配策略完成具体资源规划。通过这

一机制，系统能够把任务特征、调度目标和具体算法路径连接起来，使调度过程既具备自动化能力，也具备可解释性。

4.4 决策结果输出

自动决策机制完成场景识别和调度目标选择后，还需要将决策结果以可记录、可查询和可展示的形式输出。对于调度系统而言，仅在后端完成资源规划并不足以说明系统具备可管理性；用户还需要了解任务被识别为什么场景、系统为什么选择当前调度路径、资源是否分配成功以及任务为何进入等待状态。因此，本系统将自动决策结果与调度日志、任务状态和前端展示进行关联，使调度过程具备更强的可解释性。

决策结果输出主要包括四类信息。第一类是场景识别结果，即系统判断当前任务属于低时延任务、多资源均衡任务、GPU 辅助任务、CPU 密集型任务或依赖约束任务等哪一类场景。第二类是决策目标，即系统根据场景选择的调度方向，例如快速可行分配、低复用高质量分配、跨阵面扩展分配、计算负载均衡分配或依赖阻塞等待等。第三类是资源规划摘要，用于记录任务实际获得的阵元、CPU 和 GPU 资源信息，或者记录资源规划失败的原因。第四类是过程状态信息，用于说明任务当前处于待调度、等待、运行或完成等状态。

在后端实现中，决策结果会随着调度过程写入调度日志。调度器在执行任务场景识别、天线阵元规划、CPU/GPU 资源规划、等待处理和任务完成等关键步骤时，会将对应的动作类型、原因说明和资源分配结果保存到日志表中。这样做可以避免调度过程只存在于程序运行内存中，使任务在多轮调度中的状态变化能够被持久化记录。对于因依赖未满足、阵元规划失败、CPU 核数不足或 GPU 显存不足而无法运行的任务，系统也会将具体等待原因写入日志，便于后续查询和分析。

在前端展示层面，决策结果可以通过任务详情、状态标签和调度时间线等形式呈现。用户查看某个任务时，不仅可以看到任务是否已经运行，还可以看到系统识别出的任务场景、对应的决策目标以及等待或运行的原因。对于已经成功分配资源的任务，前端可以展示阵元、CPU 和 GPU 的分配摘要；对于未能运行的任务，则可以展示依赖阻塞、阵元不足、计算资源不足或分配失败等原因。通过这种方式，系统将后端调度逻辑转化为用户能够理解的过程信息。

决策结果输出还为后续测试与结果分析提供了数据基础。第六章中的典型任务场景验证，需要判断系统是否正确识别任务场景、是否输出了符合任务特征的决策目标，以及资源分配结果是否与预期一致。这些分析都依赖调度日志和决策输出信息。如果系统

只保存最终任务状态，而不记录场景、决策原因和资源摘要，则难以说明调度行为是否具有明确依据，也不利于对系统效果进行复盘。

综上，决策结果输出机制是连接自动决策、调度日志和前端可视化的重要环节。它使系统不仅能够完成资源分配，还能够说明资源分配过程中的判断依据和状态变化，从而增强调度过程的透明度、可追踪性和工程可信度。该机制也为下一章调度日志与动态反馈设计提供了直接基础。

4.5 本章小结

本章围绕任务场景识别与自动决策机制展开说明。首先分析了该机制的设计思路，指出系统不应要求用户手动选择具体调度算法，而应根据任务需求和当前资源状态自动判断任务所属场景，并将识别结果转换为相应的调度目标。随后，本章从约束层、资源层和策略层三个层次说明了场景识别规则，使系统能够区分低时延任务、多资源均衡任务、GPU 辅助任务、高优先级小规模任务、大规模阵元任务、CPU 密集型任务和依赖约束任务等典型场景。

在此基础上，本章进一步给出了任务场景与自动决策目标之间的映射关系。系统通过快速可行分配、公平均衡分配、单卡局部性分配、低复用高质量分配、跨阵面扩展分配、计算负载均衡分配和依赖阻塞等待等决策目标，将任务特征与第三章中的具体调度算法和资源规划策略连接起来。该机制使系统能够根据不同任务场景选择差异化的阵元搜索方式和 CPU/GPU 分配方式，而不是对所有任务采用固定调度路径。

最后，本章说明了决策结果的输出方式。系统将场景识别结果、决策目标、资源规划摘要和等待原因等信息写入调度日志，并通过前端任务详情、状态标签和调度时间线等方式进行展示。通过本章设计，系统在联合调度流程之上形成了任务理解、目标选择和结果解释能力，为下一章调度日志与动态反馈机制的设计提供了基础。

第五章 调度日志与动态反馈设计

第三章和第四章分别说明了系统如何完成天线阵元、CPU、GPU 的联合调度，以及如何根据任务特征进行场景识别和自动决策。但对于一个资源调度管理系统而言，仅给出最终分配结果并不足以支撑系统运行分析和用户理解。任务为何被调度、为何等待、为何选择某种资源规划路径，以及资源状态如何随任务运行而变化，都需要通过日志记录、状态反馈和前端展示进行持续表达。

本章围绕调度日志与动态反馈机制展开说明。首先介绍调度日志设计，说明系统如何记录调度开始、场景识别、等待原因、资源分配和任务完成等关键事件；随后分析时间片与状态反馈机制，说明系统如何在调度主循环中更新任务剩余时间、公平份额和资源占用状态；接着介绍前端可视化展示方式，说明系统如何通过任务状态、资源负载和调度时间线呈现调度过程；最后对本章内容进行总结，并为后续系统测试与结果分析提供基础。

5.1 调度日志设计

在资源调度系统中，仅保存任务的最终状态难以完整反映系统的调度行为。对于一个任务而言，用户不仅需要知道其是否运行成功，还需要了解任务何时进入调度、系统识别出何种任务场景、选择了何种调度目标、资源规划是否成功，以及任务因何进入等待状态。因此，本系统引入调度日志机制，对任务调度过程中的关键事件进行记录，使调度行为具备可追踪和可解释的能力。

调度日志主要围绕任务调度过程中的状态变化和原因说明展开。系统在任务进入调度流程时记录调度开始事件，在完成场景识别和自动决策后记录场景类型与决策目标，在资源规划失败时记录具体等待原因，在资源分配成功后记录资源分配摘要，在任务运行过程中记录状态变化，在任务完成后记录资源释放结果。通过这些日志，系统能够将一次调度行为从“输入任务需求”到“输出调度结果”的过程保存下来。

在数据层面，调度日志主要由 `t_schedule_log` 表进行持久化保存。该表以任务标识为关联核心，记录日志动作类型、详细说明和创建时间等信息。日志内容既可以用于后端排查调度异常，也可以供前端按照时间顺序展示任务调度时间线。与只保存任务状态字段相比，日志表能够保留更加细粒度的过程信息，例如任务是因为依赖未满足而等待，还是因为阵元资源不足、CPU 资源不足或 GPU 显存不足而等待。

调度日志类型如表 5-1 所示。

表 5-1 调度日志类型说明表

日志类型	含义	触发条件	作用
SCHEDULE_START	任务进入调度	调度器开始处理某个待调度任务	记录本轮调度入口
WAIT_RESOURCE	总体资源不足	阵元、CPU 或 GPU 总体容量无法满足任务需求	说明任务因资源不足等待
WAIT_COMPUTE	计算条件受阻	依赖未满足、互斥冲突或 CPU/GPU 规划失败	说明任务因计算侧或约束条件等待
WAIT_ANTENNA	阵元规划失败	无法根据当前阵元状态形成可行阵元组合	说明任务因天线侧资源受阻
WAIT_ALLOCATE	分配落库失败	候选方案可行但写入分配记录或更新状态失败	定位资源分配落地异常
RUNNING_UPDATE	运行状态更新	时间片推进, 任务剩余时间或公平份额发生变化	记录任务运行过程变化
COMPLETE	任务完成	任务剩余时间归零并完成资源释放	记录任务结束与资源释放结果

由表 5-1 可知, 调度日志并不是简单记录“成功”或“失败”, 而是对任务在不同调度阶段的状态变化进行分类描述。其中, 等待类日志是系统可解释性的重点。当任务无法启动时, 系统会根据失败位置区分总体资源不足、计算资源受阻、阵元规划失败或分配落库失败等情况, 使用户能够了解任务等待的具体原因, 而不是只看到笼统的等待状态。

调度日志还承担着连接自动决策机制与前端展示的作用。第四章中的场景识别结果和决策目标, 会在调度过程中写入日志或任务详情中; 第五章后续的前端可视化展示, 则可以基于日志数据还原任务的调度时间线。对于第六章的测试分析而言, 日志也是判断系统是否正确识别任务场景、是否选择合理调度路径、是否正确处理等待状态的重要依据。

综上, 调度日志设计使系统具备了过程记录能力。它不仅服务于系统运行时的错误排查和状态展示, 也为论文中的实验验证与结果分析提供了可查询的数据支撑。

5.2 时间片与状态反馈机制

在多任务并发场景下, 资源调度并不是一次性完成的静态过程。任务进入运行状态后, 仍然需要持续更新剩余运行时间、资源占用状态和公平份额等信息; 等待中的任务

也需要在后续调度周期中重新参与资源评估。因此，本系统在联合调度主循环中引入时间片与状态反馈机制，使任务状态能够随调度周期动态变化。

时间片机制的基本思想是将任务运行过程划分为连续的调度周期，该思想与操作系统分时调度中的时间片机制具有一定相似性^[19,20]。调度器在每一轮周期中扫描当前运行任务，对任务剩余运行时间进行递减，并根据任务资源占用情况更新其虚拟公平份额。当任务剩余时间减为零时，系统判定任务执行完成，释放其占用的天线阵元、CPU 和 GPU 资源，并将任务状态更新为完成状态。若任务尚未完成，则继续保持运行状态，并在下一轮时间片中继续更新。

状态反馈机制主要体现在任务状态、资源状态和日志记录三个方面。首先，任务状态会根据调度结果在待调度、等待、运行和完成之间转换。其次，资源状态会随着任务启动和完成而变化，例如任务运行时阵元、CPU 和 GPU 被标记为占用，任务完成后对应资源重新释放。最后，系统会在状态变化过程中写入调度日志，记录任务等待、运行更新、完成释放等关键事件，从而保证调度过程可以被追踪和复盘。

在具体实现上，`remaining_seconds` 用于描述任务距离完成还需要的剩余时间，`virtual_share` 用于描述任务在调度过程中的累计资源占用程度。前者直接决定任务是否完成，后者则用于后续调度排序和公平性控制。当系统周期性更新这两个字段时，调度器不仅能够推进任务运行过程，还能够多任务竞争场景中根据历史资源占用情况调整任务排序，避免部分任务长期占用资源而影响整体公平性。

等待任务的状态反馈同样重要。对于因依赖未满足、资源不足或规划失败而进入等待状态的任务，系统不会直接丢弃，而是在下一轮调度周期中重新检查其约束条件和资源可用情况。如果前置任务完成或资源被释放，等待任务就有机会重新进入资源规划流程。这样可以使系统适应任务持续到达、资源动态释放和多任务竞争等情况，形成较完整的动态调度闭环。

时间片与状态反馈机制还增强了前后端状态一致性。由于任务剩余时间、任务状态和资源占用信息会被持续写回数据库，前端在查询任务详情、任务列表或资源负载时，能够获得与后端调度器一致的运行视图。与此同时，系统也可以通过 `WebSocket` 将关键状态变化推送给前端，使用户能够更及时地观察任务运行和资源变化过程。

综上，时间片与状态反馈机制使系统从一次性资源分配扩展为持续运行的动态调度过程。通过周期性更新任务剩余时间、公平份额、资源占用状态和调度日志，系统能够

保持任务状态、资源状态和前端展示之间的一致性，为调度过程可解释和后续测试分析提供支撑。

5.3 前端可视化展示

调度日志和状态反馈机制最终需要通过前端界面呈现给用户。对于数字阵一体化资源调度管理系统而言，前端展示的作用不仅是提供操作入口，还需要将资源状态、任务状态、调度原因和运行过程以直观方式表达出来，使用户能够理解系统当前运行情况以及任务调度结果。

系统前端主要通过 REST API 和 WebSocket 两种方式获取调度相关信息。REST API 主要用于查询任务列表、任务详情、资源状态和调度日志等数据，适合处理常规页面加载和用户主动查询；WebSocket 则用于接收任务状态变化、资源告警和调度结果等实时信息，适合展示调度过程中的动态变化。通过二者结合，前端既能够保证数据查询的稳定性，也能够提高调度反馈的实时性。

在资源状态展示方面，前端首先提供天线资源状态展示界面。该界面以资源地图的形式展示物理天线、天线单元、活跃单元和未绑定父级单元等统计信息，并通过物理天线列表展示各天线所属阵面、在线状态和包含的天线单元数量。该界面能够帮助用户直观了解天线资源的整体规模和当前状态，为任务调度中的阵元资源判断提供可视化依据，如图 5-1 所示。

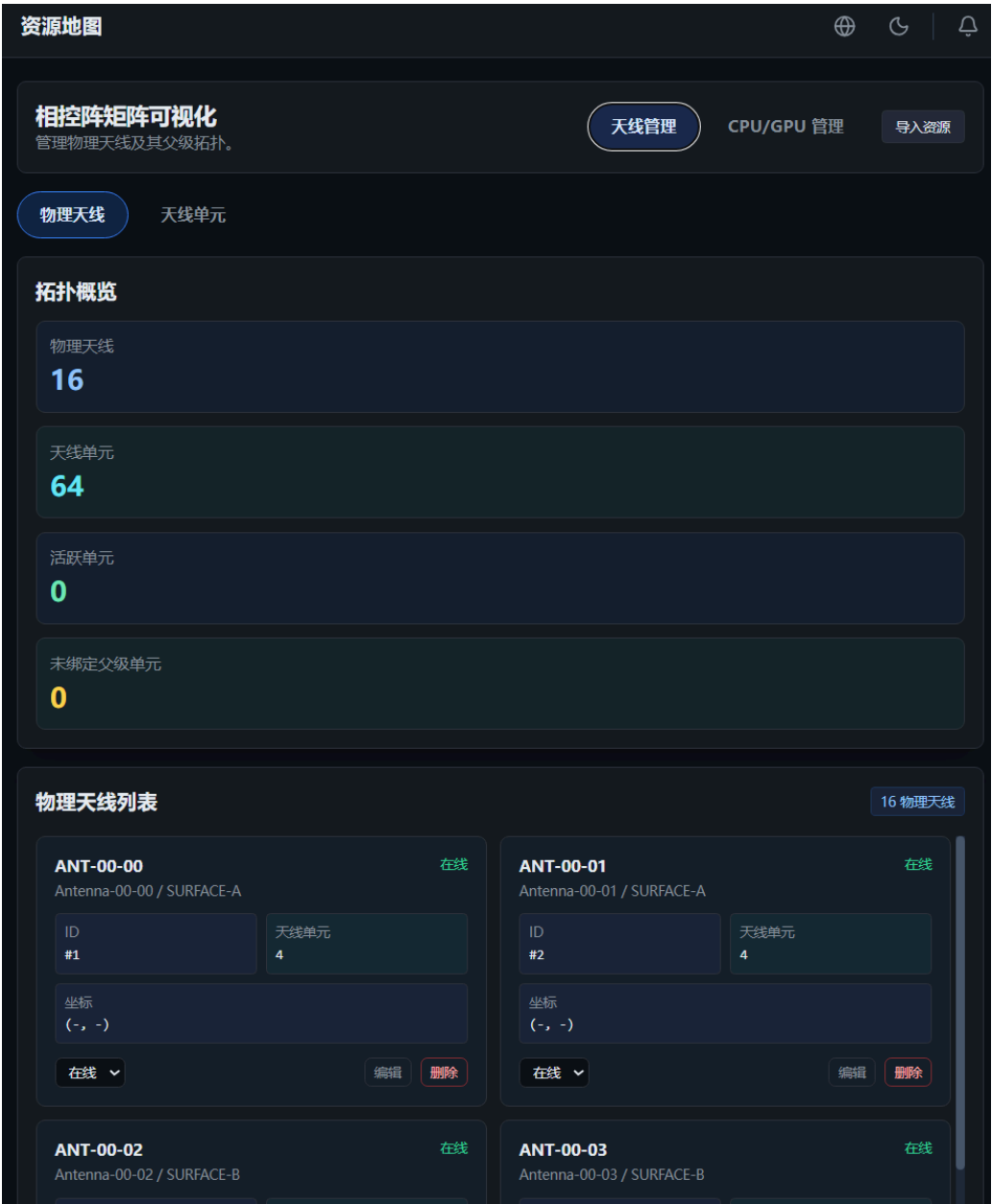


图 5-1 天线资源状态展示界面

除天线阵元资源外，系统还提供 CPU/GPU 计算资源展示界面。该界面将 CPU 节点和 GPU 节点分区展示，能够显示计算节点数量、节点 IP、核心数、GPU 型号、显存容量和在线状态等信息。通过该界面，用户可以快速查看当前计算资源的整体负载和可用情况，为理解 CPU 密集型任务、GPU 辅助任务的调度结果提供支撑，如图 5-2 所示。

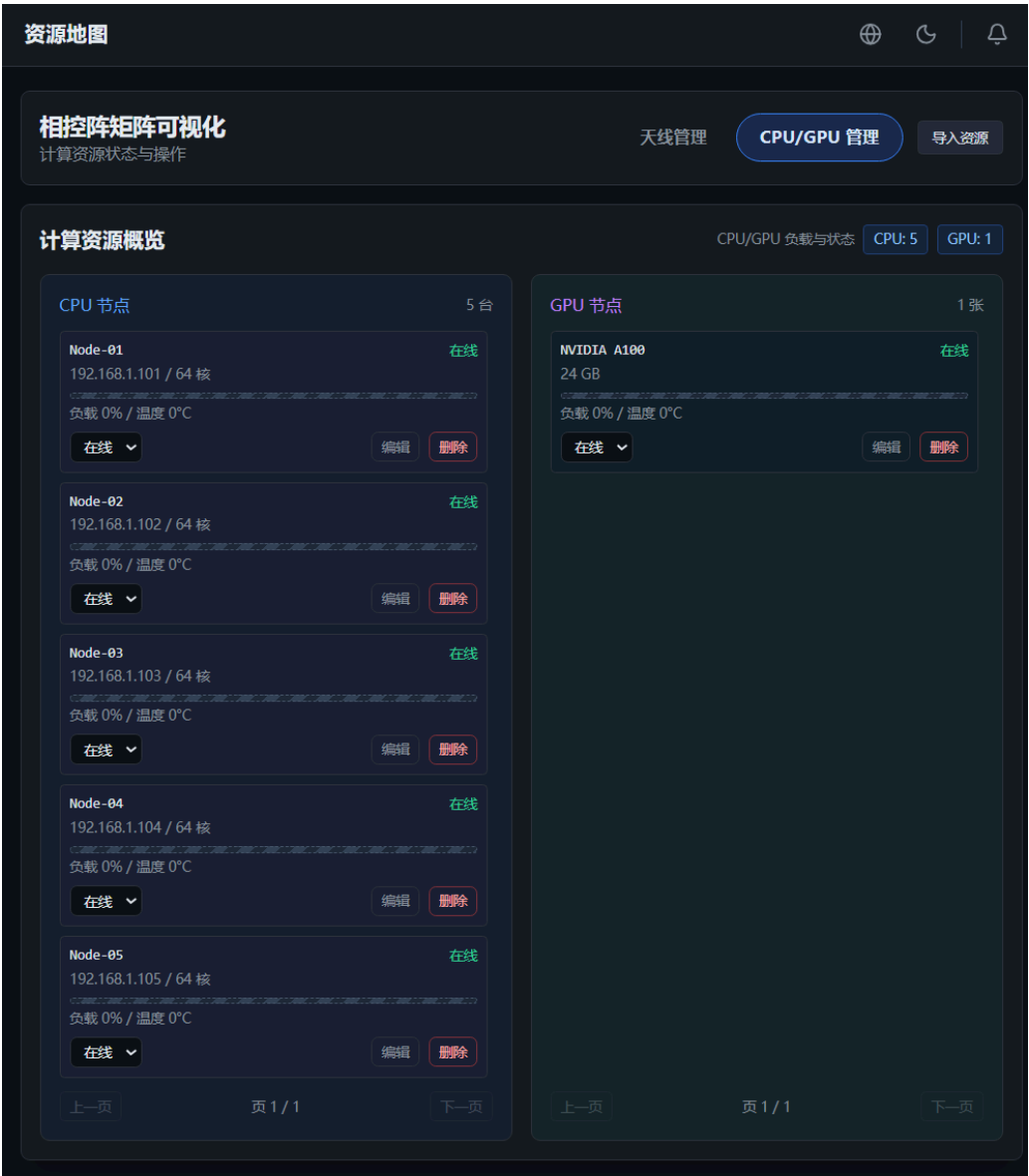


图 5-2 CPU/GPU 资源状态展示界面

在任务状态展示方面，前端通过任务列表和任务详情区域展示任务名称、优先级、资源需求、当前状态、剩余时间和调度结果等信息。对于处于等待状态的任务，系统可以展示等待原因，例如依赖未满足、阵元规划失败、CPU 资源不足或 GPU 显存不足；对于已经运行的任务，系统可以展示实际分配到的阵元、CPU 和 GPU 资源摘要；对于已完成任务，系统可以展示完成时间和资源释放结果。通过这些信息，用户能够快速判断任务所处阶段及其调度结果。

调度时间线是前端可视化中的重要组成部分。系统可以按照时间顺序展示任务从提交、进入调度、等待、运行、状态更新到完成释放的全过程。时间线中的每个节点对应一条或多条调度日志，能够说明该阶段发生了什么事件以及系统给出了什么原因说明。与

单纯展示最终状态相比，时间线能够更完整地反映调度过程，使用户理解任务状态变化的来龙去脉。

任务调度详情界面还能够展示第四章中的场景识别结果和自动决策目标。例如，任务被识别为 GPU 辅助任务时，界面可以展示对应的 GPU 分配结果、CPU 分配方案、阵面选择结果以及决策原因。用户不需要了解后端具体调用了哪一种搜索算法，也可以通过解释性文本理解系统为什么采用当前调度路径。如图 5-3 所示，系统在任务详情中展示了 CPU 方案、GPU 卡、阵面信息、决策原因和调度时间线，体现了调度过程的可解释性。



图 5-3 任务调度时间线与决策结果展示界面

从工程意义上看，前端可视化展示增强了系统的可用性和可解释性。一方面，资源负载、任务状态和调度时间线能够降低用户理解系统运行状态的成本；另一方面，等待原因、决策原因和资源分配摘要能够帮助用户分析任务未运行或调度失败的原因。对于毕业设计展示和测试分析而言，前端界面也能够更加直观地证明系统已经形成了从任务提交、自动调度、过程记录到结果展示的完整闭环。

综上，前端可视化展示是调度日志和动态反馈机制的外部表达。它将后端调度过程中产生的任务状态、资源状态、决策原因和日志信息转化为用户可理解的界面内容，使系统不仅能够完成调度计算，还能够展示调度过程、解释调度结果，并支撑后续系统测试与答辩演示。

5.4 本章小结

本章围绕调度日志与动态反馈机制展开说明。首先介绍了调度日志设计，说明系统如何通过 `t_schedule_log` 表记录任务进入调度、资源等待、阵元规划失败、计算资源受阻、运行状态更新和任务完成等关键事件，使调度过程不再只体现为最终状态，而是具备可查询、可追踪和可解释的过程记录。

随后，本章分析了时间片与状态反馈机制。系统通过周期性调度循环更新任务剩余时间、虚拟公平份额和资源占用状态，并在任务完成后释放天线阵元、CPU 和 GPU 资源，使等待任务能够在后续调度周期中重新参与资源评估。该机制保证了任务状态、资源状态和日志记录之间的一致性。

最后，本章介绍了前端可视化展示方式。系统通过资源地图、CPU/GPU 资源状态界面和任务调度时间线，将后端调度过程中的资源状态、决策原因、等待原因和分配结果呈现给用户。通过本章设计，系统形成了“日志记录—状态反馈—前端展示”的动态反馈闭环，为第六章系统测试与结果分析提供了过程数据和展示支撑。

第六章 系统测试与结果分析

前文已经从系统总体设计、联合调度算法、场景识别决策以及调度日志反馈等方面，对数字阵一体化资源调度管理系统进行了设计与说明。为了验证系统设计是否能够在实际运行过程中发挥作用，还需要通过典型测试场景对系统功能和调度结果进行检验。系统测试不仅用于确认前后端功能是否正常，也用于验证天线阵元、CPU 和 GPU 三类资源能否按照任务需求完成联合调度。

本章围绕系统测试与结果分析展开。首先介绍测试目的、测试环境和资源配置，明确测试所依赖的软件环境与模拟资源规模；随后设计低时延任务、多资源均衡任务、GPU 辅助任务、CPU 密集型任务和依赖约束任务等典型场景，用于覆盖系统中的主要调度路径；接着对测试结果进行分析，重点验证场景识别结果、自动决策目标、资源分配结果和等待原因是否符合预期；最后总结系统在功能完整性、工程可用性和当前实现不足等方面的表现，为全文结论提供实验依据。

6.1 测试目的与测试环境

为了验证数字阵一体化资源调度管理系统的功能完整性和调度有效性，本章围绕典型任务场景开展系统测试与结果分析。测试并不只关注接口是否能够正常调用，而是重点验证系统能否在多任务条件下完成资源识别、联合调度、状态更新、日志记录和前端展示等完整流程。

本系统测试目的主要包括以下几个方面。第一，验证系统是否能够完成任务提交、任务入库、资源规划、任务运行和资源释放等基本流程；第二，验证天线阵元、CPU 和 GPU 三类资源是否能够在同一调度流程中协同分配；第三，验证任务场景识别与自动决策机制是否能够根据不同任务特征输出合理的调度目标；第四，验证依赖未满足、资源不足或资源规划失败等异常场景下，系统是否能够保持任务等待状态并记录明确原因；第五，验证调度日志、任务状态和前端时间线是否能够正确反映调度过程。

从测试内容看，本章重点关注系统的调度正确性、场景适配性和工程可用性。调度正确性主要考察任务获得的阵元、CPU 和 GPU 资源是否满足任务需求；场景适配性主要考察低时延任务、GPU 辅助任务、CPU 密集型任务和依赖约束任务等不同场景是否触发对应的调度目标；工程可用性则主要考察系统前后端交互、接口鉴权、日志记录和可视化展示是否能够支撑完整使用流程。

本系统测试环境主要由前端、后端、数据库和模拟资源数据组成。后端基于 Spring Boot 多模块结构实现，负责认证鉴权、资源管理、任务管理、调度控制和日志记录；前端基于 React 与 Vite 实现，负责资源状态展示、任务提交、调度结果展示和时间线可视化；数据库采用 MySQL，用于保存用户、资源、任务、分配记录和调度日志等数据。测试过程中，通过构造不同类型的任务请求，观察系统生成的场景识别结果、资源分配结果、任务状态变化和日志记录情况。

测试环境与资源配置如表 6-1 所示。

表 6-1 测试环境与资源配置表

项目	配置或数量	说明
后端框架	Spring Boot	采用多模块结构，包含认证、资源、任务和调度等模块
前端框架	React + Vite	用于实现资源展示、任务调度、状态反馈和时间线可视化
数据库	MySQL	保存用户信息、资源信息、任务信息、分配记录和调度日志
物理天线	16 个	用于承载天线阵元资源的物理层对象
天线单元	64 个	调度过程中的细粒度阵元资源
阵面数量	4 个	SURFACE-A 至 SURFACE-D，用于验证单阵面优先与跨阵面分配
CPU 节点	5 台	用于验证紧凑式 CPU 分配和负载均衡式 CPU 分配
GPU 节点	1 张	用于验证 GPU 辅助任务和单卡局部性分配
测试方式	典型任务构造	通过提交不同资源需求和约束条件的任务，验证系统调度行为

由表 6-1 可知，当前测试环境虽然不属于真实大规模生产环境，但已经覆盖了本系统需要验证的关键资源类型。64 个天线单元和 4 个阵面能够支撑单阵面分配、跨阵面分配和大规模阵元任务测试；5 个 CPU 节点能够验证普通任务的紧凑式分配和 CPU 密集型任务的负载均衡分配；GPU 节点则能够验证 GPU 辅助任务的单卡局部性分配逻辑。因此，该测试环境能够满足毕业设计阶段对系统功能、调度策略和可解释性验证的要求。

6.2 测试场景设计

为验证系统在不同任务特征下的调度表现，本节围绕典型数字阵任务场景构造测试用例。测试场景设计需要覆盖天线阵元、CPU 和 GPU 三类资源的主要约束，同时也要

包含依赖关系、资源不足和多任务竞争等情况，以便观察系统是否能够正确完成场景识别、自动决策和联合资源分配。

结合第三章和第四章的调度设计，本文选取低时延任务、多资源均衡任务、GPU 辅助任务、CPU 密集型任务和依赖约束任务作为主要测试场景。其中，低时延任务用于验证系统是否能够优先选择快速可行路径；多资源均衡任务用于验证系统是否能够在阵元和计算资源之间取得相对平衡；GPU 辅助任务用于验证单卡局部性分配是否生效；CPU 密集型任务用于验证多个 CPU 节点之间的负载均衡能力；依赖约束任务用于验证系统在前置条件不满足时是否能够正确阻塞任务并记录等待原因。

测试场景设计如表 6-2 所示。

表 6-2 测试场景设计表

测试任务	任务特征	预期场景	验证重点
IT_AUTO_LOW_LATENCY_VERIFY	阵元需求小、CPU 需求低、截止时间较短	低时延任务	是否优先选择快速可行路径并快速启动任务
IT_AUTO_MEDIUM_VERIFY	中等规模阵元需求，伴随一定 CPU 需求	多资源均衡任务	是否兼顾资源利用率、解质量和公平性
IT_AUTO_GPU_VERIFY	存在 GPU 显存需求，CPU 需求中等	GPU 辅助任务	是否优先选择单张 GPU 满足显存需求
IT_AUTO_CPU_VERIFY	CPU 核数需求较高，阵元需求相对较小	CPU 密集型任务	是否将 CPU 需求分散到多个节点以降低负载热点
IT_AUTO_DEPENDENCY_VERIFY	存在未满足依赖任务	依赖约束任务	是否停止资源规划并记录依赖阻塞等待原因

由表 6-2 可知，测试任务并不是简单验证接口返回是否成功，而是分别对应系统中的关键调度场景。低时延任务和多资源均衡任务主要用于验证天线阵元搜索策略与普通 CPU 分配是否能够正常配合；GPU 辅助任务用于验证系统在存在 GPU 需求时是否能够执行单卡局部性分配；CPU 密集型任务用于观察负载均衡式 CPU 分配是否能够落实到真实分配结果；依赖约束任务则用于验证系统不会在前置条件缺失时错误启动任务。

在测试执行过程中，每类任务均通过前端或测试脚本提交至系统，任务进入数据库后由调度器在下一轮调度周期中自动处理。系统完成调度后，分别查看任务状态、调度日志、资源分配记录和前端展示结果，并与预期场景和预期决策目标进行比对。若任务

被正确识别为对应场景，且调度结果或等待原因与任务特征一致，则说明系统在该场景下的调度行为符合设计预期。

上述测试场景能够覆盖系统的主要调度路径，但并不代表系统在所有可能任务组合下都已充分验证。由于毕业设计阶段的测试环境规模有限，本章更侧重于通过典型任务验证系统核心机制是否成立，包括场景识别、资源联合分配、等待处理、日志记录和可视化反馈等关键能力。

6.3 测试结果与分析

根据前述测试场景，系统分别对低时延任务、多资源均衡任务、GPU 辅助任务、CPU 密集型任务和依赖约束任务进行了调度验证。测试过程中主要观察任务的场景识别结果、自动决策目标、资源分配结果和调度日志记录情况，以判断系统调度行为是否符合预期。

典型场景测试结果如表 6-3 所示。

表 6-3 典型场景测试结果表

测试任务	场景识别结果	决策目标	主要分配结果
IT_AUTO_LOW_LATENCY_VERIFY	LOW_LATENCY	FAST_FEASIBLE	algorithm=BFS, surface=SURFACE-A, cpu=CPU-6:1
IT_AUTO_MEDIUM_VERIFY	BALANCED_MULTI_RESOURCE	FAIR_BALANCED	algorithm=DIJKSTRA, surface=SURFACE-B, cpu=CPU-6:1
IT_AUTO_GPU_VERIFY	GPU_ACCELERATED	GPU_LOCALITY_SINGLE_CARD	algorithm=BFS, surface=SURFACE-A, gpu=GPU-9:4
IT_AUTO_CPU_VERIFY	CPU_INTENSIVE	COMPUTE_LOAD_BALANCE	algorithm=BFS, surface=SURFACE-C, cpu=CPU-6:8, CPU-7:8, CPU-8:8, CPU-9:8, CPU-10:8
IT_AUTO_DEPENDENCY_DEPENDENCY_VERIFY	DEPENDENCY_CONSTRAINED	DEPENDENCY_BLOCKED	WAIT_COMPUTE, 记录依赖阻塞原因

由表 6-3 可知，系统能够根据不同任务特征识别出对应的任务场景，并输出相应的自动决策目标。低时延任务被识别为 LOW_LATENCY，系统选择 FAST_FEASIBLE 作为调度目标，并采用 BFS 快速搜索路径完成阵元分配，说明系统能够在小规模、时限较紧的任务中优先降低搜索开销，使任务较快进入运行状态。

多资源均衡任务被识别为 `BALANCED_MULTI_RESOURCE`，系统输出 `FAIR_BALANCED` 决策目标，并在阵元侧选择低权重路径搜索结果。该结果说明系统在不存在明显单一资源瓶颈的场景下，并不会只追求最快得到可行解，而是会兼顾阵元复用、分配质量和资源公平性，使调度结果更符合多资源均衡场景的设计目标。

GPU 辅助任务被识别为 `GPU_ACCELERATED`，系统输出 `GPU_LOCALITY_SINGLE_CARD` 决策目标，并为任务分配单张 GPU 资源。该结果表明系统能够识别任务中的 GPU 显存需求，并按照第四章中设计的单卡局部性原则进行分配，避免将一个任务的显存需求拆分到多张 GPU 上，从而降低 GPU 资源管理和释放过程的复杂度。

CPU 密集型任务被识别为 `CPU_INTENSIVE`，系统输出 `COMPUTE_LOAD_BALANCE` 决策目标，并将 CPU 核数需求分散到多个 CPU 节点上。测试结果中，任务所需 CPU 资源被分配到 CPU-6 至 CPU-10 五个节点，每个节点分配 8 核，说明系统的负载均衡式 CPU 分配已经落实到实际分配结果中，而不是仅停留在场景标签或决策说明层面。这一结果验证了系统在 CPU 高需求任务下避免单节点负载过高的能力。

依赖约束任务被识别为 `DEPENDENCY_CONSTRAINED`，系统输出依赖阻塞等待结果，并将任务保持在等待状态。该场景下，系统没有继续执行天线阵元和 CPU/GPU 资源规划，而是直接记录 `WAIT_COMPUTE` 等待原因，说明系统能够在前置条件不满足时阻止任务错误启动。该结果验证了系统对依赖关系和任务运行约束的处理能力。

从调度正确性角度看，测试结果表明系统能够在不同任务场景下触发差异化的调度行为。低时延任务对应快速可行路径，GPU 辅助任务对应单卡 GPU 分配，CPU 密集型任务对应多节点均衡分配，依赖约束任务对应阻塞等待。不同任务的场景识别结果、决策目标和资源分配结果之间保持一致，说明系统的“场景识别—自动决策—联合调度”链路能够正常工作。

从资源分配结果看，系统能够同时处理阵元、CPU 和 GPU 三类资源。任务成功运行前，系统会先生成阵元候选方案和计算资源候选方案，只有当多类资源均满足条件时才写入分配记录并更新任务状态。测试中未出现只分配单类资源而任务整体无法运行的情况，说明第三章中“先规划、后落库”的联合调度思想在测试场景中得到了验证。

从调度可解释性角度看，系统能够通过调度日志保存场景识别结果、决策目标、资源分配摘要和等待原因。对于成功运行的任务，日志能够反映其进入调度、生成资源方案和切换为运行状态的过程；对于等待任务，日志能够说明其等待原因。结合第五章中的前端时间线展示，用户可以较清楚地理解任务为何运行、为何等待以及资源如何分配。

综合上述结果可以看出，本系统已经能够在典型任务场景下完成任务识别、自动决策、资源分配、等待处理和日志记录等核心流程。虽然当前测试规模仍属于毕业设计阶段的模拟环境，但测试结果能够说明系统设计具备基本可行性，能够支撑数字阵多资源联合调度管理的主要需求。

6.4 调度算法性能量化对比

前述测试验证了系统能够根据不同任务场景自动选择不同调度路径。然而，仅验证系统选择了正确的路径尚不足以说明各算法本身的性能差异。为深入分析各调度算法的实际表现，本节对天线调度和计算资源调度进行量化对比实验，从执行效率、解质量和可行性三个维度对算法性能进行评价。

6.4.1 天线调度算法执行时间对比

实验在相同资源状态下，对每个测试任务分别强制使用 BFS、Dijkstra、Greedy、Heap 和 DP 五种算法进行阵元分配规划（不实际写入分配结果），记录各算法的执行时间。阵元需求量设定为 4、8、12、16、24、32、48 个阵元，每个配置运行 5 次取中位数。

天线调度算法执行时间对比结果如表 6-4 所示。

表 6-4 天线调度算法执行时间对比（单位： μs ）

阵元需求	BFS	Dijkstra	Greedy	Heap	DP
4	2102	2178	2054	5442	1782
8	1969	1648	6445	5078	1622
12	1458	1677	10144	4685	1439
16	1892	1646	15744	4592	1628
24	1453	1232	26418	4319	1292
32	1488	1273	39118	4688	1807
48	1756	1989	60593	4809	1310

由表 6-4 可知，BFS、Dijkstra 和 DP 三种算法的执行时间基本不随阵元需求增长而变化，始终维持在 1.2~2.2 ms 范围内，表现出较好的时间稳定性。DP 算法的稳定性来源于其候选集上限约束，当候选阵元数超过阈值时算法直接返回不可行结果，计算量不会无限增长。Greedy 算法的执行时间随阵元需求呈近似线性增长，从 4 阵元时的约 2 ms 增长到 48 阵元时的约 60 ms，符合贪心搜索逐步扩展前沿的时间复杂度特征。Heap 算法的执行时间相对稳定在 4~5 ms 范围内，高于 BFS/Dijkstra/DP 但显著低于 Greedy，这是因为 Heap 采用多阵面游标交错选取的策略，每步仅从优先队列中取出一个最优候选，避免了全量遍历。

各算法执行时间随阵元需求的变化趋势如图 6-1 所示。

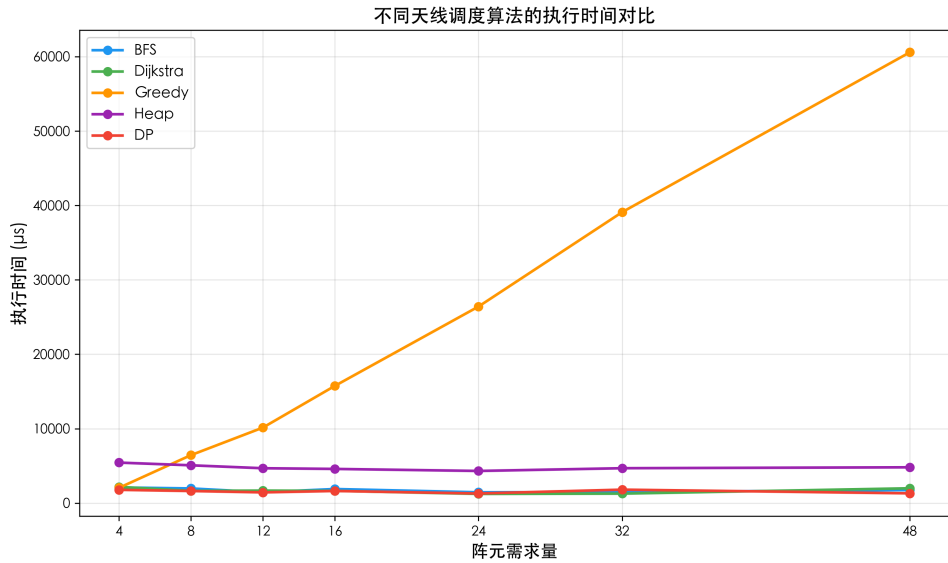


图 6-1 不同天线调度算法的执行时间对比

6.4.2 天线调度算法可行性对比

除执行时间外，各算法在不同阵元需求下能否获得可行解是另一个关键评价指标。

可行性对比结果如表 6-5 所示。

表 6-5 天线调度算法可行解获取能力对比

阵元需求	BFS	Dijkstra	Greedy	Heap	DP
4	可行	可行	可行	可行	可行
8	不可行	不可行	可行	可行	不可行
12	不可行	不可行	可行	可行	不可行
16	不可行	不可行	可行	可行	不可行
24	不可行	不可行	可行	可行	不可行
32	不可行	不可行	可行	可行	不可行
48	不可行	不可行	可行	可行	不可行

由表 6-5 可知，Greedy 和 Heap 在全部测试规模下均能获得可行解，表现出最强的可行性。Greedy 通过逐步扩展前沿并允许跨阵面选择保证了搜索范围的充分性；Heap 通过多阵面游标交错选取天然具备跨阵面搜索能力。BFS 和 Dijkstra 在阵元需求超过 4 时即失去可行性，这是因为两者的搜索严格限制在同阵面邻接关系内，当单阵面连通区域不足以满足需求时无法跨阵面扩展。DP 受限于候选集上限，在当前邻接约束下难以构造满足需求的连通集合。

需要说明的是，上述基准测试采用强制单算法执行的方式，目的是评估每种算法的独立能力上限。在系统实际运行中，调度器并非只尝试单一算法，而是通过第四章所述的回退机制依次尝试多种算法：若主选算法未获得可行解，系统会自动切换到下一候选算法继续搜索。因此，BFS、Dijkstra 和 DP 在大规模场景下的“不可行”结果并不意味着系统整体调度失败，而是说明这些算法在单独使用时不适合处理大规模跨阵面任务。系统通过自动回退机制确保最终总能由 Greedy 或 Heap 兜底完成分配。这一设计思路正是“快速算法优先尝试、通用算法兜底保障”的分层调度策略。

上述结果验证了系统自动选择调度路径的必要性：没有一种算法能在所有场景下同时满足效率和可行性要求，系统需要根据任务规模自动切换算法。各算法可行解获取能力的直观对比如图 6-2 所示。

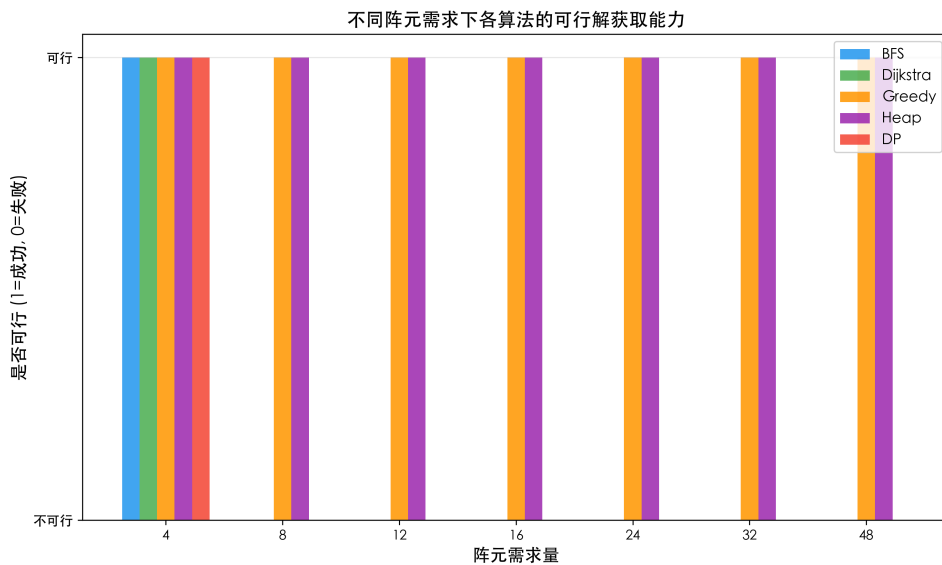


图 6-2 不同阵元需求下各算法的可行解获取能力

6.4.3 天线调度算法解质量对比

对于获得可行解的情况，通过调度得分（综合考虑阵元复用次数、空间紧凑度和跨阵面惩罚，得分越低表示方案质量越高）评价各算法的解质量。解质量对比结果如表 6-6 所示。

表 6-6 天线调度算法解质量对比（调度得分，越低越优）

阵元需求	BFS	Dijkstra	Greedy	Heap
4	10	10	10	15080
8	—	—	30	15085
12	—	—	50	15090
16	—	—	5070	15090
24	—	—	5090	15110
32	—	—	5090	15110
48	—	—	15110	15130

调度得分的物理含义如下：得分由活跃复用惩罚（权重 10000）、历史复用惩罚（权重 1000）、空间展布距离和跨阵面切换惩罚（每跨一个阵面加 5000）四项累加构成。得分为 10~50 表示所选阵元几乎无复用且分布在单阵面内；得分约 5000 表示方案跨越了一个阵面边界；得分约 15000 表示方案跨越了两个以上阵面。在实际业务中，得分低于 5000 的方案通常可直接接受，5000~15000 的方案需要权衡跨阵面通信开销，超过 15000 则说明资源紧张、需要关注阵面间协调。

由表 6-6 可知，在小规模（4 阵元）场景下，BFS、Dijkstra 和 Greedy 的得分相同（均为 10），说明各算法均能找到质量一致的最优解——所选阵元无复用且集中在单阵面内。Greedy 在中等规模（8~12 阵元）时得分为 30~50，仍处于单阵面内高质量解的范围，显著优于 Heap 的约 15000（对应跨多阵面方案）。Heap 的得分始终较高，这是因为其跨阵面游标交错选取策略虽然保证了可行性，但必然引入跨阵面惩罚。随着阵元需求增大到 16 及以上，Greedy 的得分也跳升至约 5000，说明此时 Greedy 也不得不跨阵面搜索，质量优势逐渐缩小。各算法解质量随阵元需求变化的对比如图 6-3 所示。

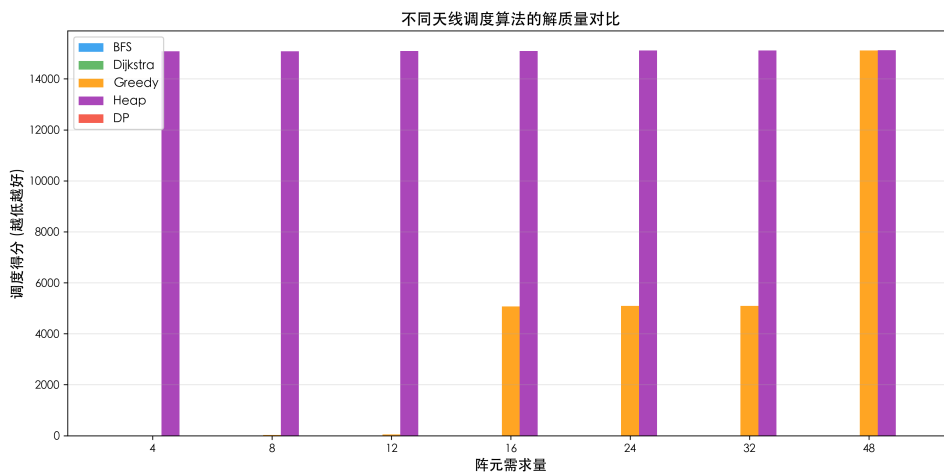


图 6-3 不同天线调度算法的解质量对比

6.4.4 天线调度算法综合评价

综合执行时间、可行性和解质量三个维度,可以对五种天线调度算法进行如表 6-7 所示的总结。

表 6-7 天线调度算法综合性能评价

算法	执行速度	可行性	解质量	适用场景
BFS	快 (1.7ms)	仅小规模	高	低时延单阵面小任务
Dijkstra	快 (1.6ms)	仅小规模	高	中等规模单阵面任务
DP	快 (1.5ms)	受限	最高	高优先级精确质量场景
Greedy	慢 (线性增长)	全规模	中高	通用跨阵面任务
Heap	中等 (4.6ms)	全规模	中	大规模跨阵面任务

该结果证明了系统根据任务特征自动选择算法的设计合理性:对低时延小任务选择 BFS 可实现最快响应;对高优先级精确调度场景选择 DP 可获得最优解质量;对大规模跨阵面场景选择 Greedy 或 Heap 可保证可行性。

6.4.5 CPU 调度模式负载均衡对比

本节对 CPU 调度的紧凑模式 (PACKED) 和均衡模式 (BALANCE) 进行量化对比。实验在 5 个 CPU 节点 (每节点 64 核) 的环境下进行,设定 CPU 需求量梯度为 4、8、16、32、64、128 核,分别使用两种模式进行资源规划,记录各节点负载方差作为均衡性指标。负载方差定义为所有节点预测负载率的方差,方差越低表示负载越均衡。

CPU 调度模式负载方差对比如表 6-8 所示。

表 6-8 CPU 调度模式负载方差对比

CPU 需求	PACKED 方差	BALANCE 方差	方差降幅
4 核	3.91×10^{-5}	3.91×10^{-5}	0%
8 核	3.52×10^{-4}	5.86×10^{-5}	83.4%
16 核	4.73×10^{-3}	3.91×10^{-5}	99.2%
32 核	2.85×10^{-2}	5.86×10^{-5}	99.8%
64 核	1.36×10^{-1}	3.91×10^{-5}	99.97%
128 核	2.22×10^{-1}	5.86×10^{-5}	99.97%

由表 6-8 可知,在小规模需求 (4 核) 下两种模式的方差完全一致,因为此时仅需分配到单个节点,无论采用何种策略结果相同。在中等规模需求 (8~32 核) 下, BALANCE 模式的方差较 PACKED 降低了 83%~99.8%,说明均衡模式能够有效地将负载分散到多个节点上。在大规模需求 (64~128 核) 下, PACKED 模式的方差急剧上升至 0.13~0.22,意味着部分节点接近满载而其余节点几乎空闲;而 BALANCE 模式的方差始终维持在 10^{-5}

量级，表明所有节点的负载几乎完全均匀。以 128 核需求为例，两种策略的方差相差约 3790 倍。

两种模式负载方差随 CPU 需求变化的对比趋势如图 6-4 所示。

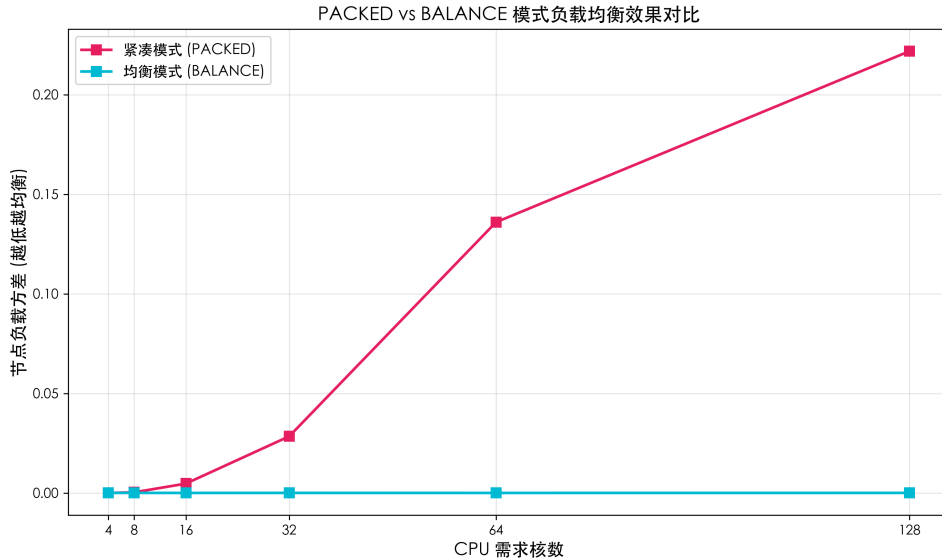


图 6-4 PACKED 与 BALANCE 模式负载均衡效果对比

实验结果证明了系统根据任务特征自动选择 CPU 调度模式的合理性。对于低时延、小规模 CPU 需求的任务，PACKED 模式虽然不如 BALANCE 均衡，但绝对方差很小，且紧凑分配减少了跨节点调度开销，适合快速启动场景。对于 CPU 密集型任务（ ≥ 32 核），系统自动切换为 BALANCE 模式，方差可降低至 PACKED 的 $1/3790$ ，有效避免了单节点过载导致的性能瓶颈。

6.4.6 多任务并发调度性能验证

前述实验均针对单任务场景进行算法性能分析。为验证系统在多任务并发条件下的实际调度能力，本节设计了一组混合工作负载实验：同时提交 20 个不同类型的任务，观察系统在资源竞争条件下的调度延迟、完成率和场景识别正确性。

实验中的 20 个任务包括：低时延小任务 5 个、中等规模多资源任务 5 个、GPU 辅助任务 4 个、CPU 密集型任务 4 个和依赖约束任务 2 个（依赖一个不存在的前置任务）。所有任务在同一时刻写入数据库，由调度器在后续调度周期中自动处理。

多任务并发调度结果如表 6-9 所示。

表 6-9 多任务并发调度结果统计

任务类型	数量	完成数	平均调度延迟	完成率
低时延任务	5	5	10.2 s	100%
GPU 辅助任务	4	4	18.0 s	100%
中等规模任务	5	5	18.6 s	100%
CPU 密集型任务	4	4	26.8 s	100%
依赖约束任务	2	0	—（阻塞）	0%（预期）
合计	20	18	17.9 s	90%(有效100%)

由表 6-9 可知，在 20 个任务同时提交的条件下，系统在 37 秒内完成了全部 18 个可调度任务的分配和执行（调度吞吐量约 0.49 任务/秒）。2 个依赖约束任务因前置条件不满足而正确保持等待状态，符合预期行为。

从调度延迟分布来看，低时延任务的平均调度延迟最短（10.2 秒），说明系统在多任务竞争条件下仍能优先处理高时间敏感度的任务。CPU 密集型任务的调度延迟最长（26.8 秒），这符合预期——CPU 密集型任务消耗大量核心资源，需要等待前序任务释放资源后才能获得足够的计算资源。调度延迟的梯度分布（低时延 < GPU < 中等 < CPU 密集）与系统基于 DRF 公平份额和优先级的排序机制一致，说明调度器在资源竞争条件下能够维持合理的任务处理顺序。

从场景识别角度看，系统在并发条件下对 18 个成功调度的任务均正确识别了对应场景（LOW_LATENCY 5 个、BALANCED_MULTI_RESOURCE 5 个、GPU_ACCELERATED 4 个、CPU_INTENSIVE 4 个），场景识别准确率为 100%，说明多任务并发并未影响场景识别机制的正确性。

6.4.7 算法可扩展性验证

为进一步评估各算法在更大资源规模下的性能趋势，本节将阵元资源从 64 个扩展到 128 个（每阵面 32 个），并在相同实验条件下重新测量各算法的执行时间和可行性。

128 阵元环境下各算法的执行时间如表 6-10 所示。

表 6-10 128 阵元环境下天线调度算法执行时间（单位： μs ）

阵元需求	BFS	Dijkstra	Greedy	Heap	DP
4	5089	3369	5532	30232	2680
8	3015	2635	11222	28784	2502
16	4137	3621	27908	28879	2362
32	3261	2874	90292	28712	2522
48	3811	2855	179926	30140	2187
64	3445	3230	270201	28563	2268

128 阵元环境下各算法的可行性如表 6-11 所示。

表 6-11 128 阵元环境下天线调度算法可行性对比

阵元需求	BFS	Dijkstra	Greedy	Heap	DP
4	可行	可行	可行	可行	可行
8	可行	可行	可行	可行	可行
16	可行	可行	可行	可行	不可行
32	不可行	不可行	可行	可行	不可行
48	不可行	不可行	可行	可行	不可行
64	不可行	不可行	可行	可行	不可行

将 64 阵元和 128 阵元两组实验进行对比，可以得出以下可扩展性结论：

第一，资源规模扩大后，BFS 和 Dijkstra 的可行性显著改善。在 64 阵元环境下两者仅在需求为 4 时可行，而在 128 阵元环境下可行范围扩展到 16 阵元。这是因为单阵面容量从 16 增长到 32，使得邻接连通区域更充分，BFS/Dijkstra 的同阵面搜索策略获得了更大的可行空间。

第二，Greedy 算法的执行时间增长最为显著。在 64 阵元需求下，Greedy 从 64 阵元环境的约 60 ms 增长到 128 阵元环境的约 270 ms，增幅约 4.5 倍。这反映了 Greedy 的时间复杂度对候选集规模敏感——候选阵元翻倍后，每步的前沿遍历代价也随之增大。

第三，BFS、Dijkstra、Heap 和 DP 的执行时间增幅均在 2 倍以内。BFS/Dijkstra 从约 1.5 ms 增长到约 3.3 ms，Heap 从约 4.6 ms 增长到约 29 ms，DP 基本不变（受限于候选集上限）。这说明除 Greedy 外的算法均具有较好的可扩展性。

第四，综合可行性和执行时间来看，系统的自动选择策略在 128 阵元规模下仍然有效：小规模任务可由 BFS/Dijkstra 快速完成（3 ms），大规模任务由 Greedy/Heap 兜底（Heap 约 29 ms，远优于 Greedy 的 270 ms）。Heap 在大规模环境下的时间优势相比 Greedy 更加突出，进一步验证了系统对大规模跨阵面任务选择 Heap 算法的合理性。

6.5 系统可用性与不足分析

从测试结果来看，本系统已经能够完成数字阵一体化资源调度管理的主要功能。在资源管理方面，系统能够对天线阵元、CPU 和 GPU 等资源进行统一展示和状态维护，使用户可以通过 Web 页面查看资源规模、在线状态和占用情况。在任务调度方面，系统能够接收带有阵元、CPU、GPU、优先级和依赖关系等约束的任务请求，并在后端调度器中完成场景识别、资源规划、任务状态更新和日志记录。测试结果表明，系统已经形成了从任务提交到资源释放的完整闭环。

在调度可用性方面，系统能够针对不同任务场景表现出差异化行为。低时延任务能够触发快速可行分配路径，GPU 辅助任务能够触发单卡局部性分配，CPU 密集型任务能够触发多节点负载均衡分配，依赖约束任务能够在前置条件未满足时进入等待状态并记录阻塞原因。这说明系统不是简单地按照固定顺序分配资源，而是能够结合任务特征和资源状态选择相对合理的调度目标。

在过程可解释性方面，系统通过调度日志和前端时间线记录任务状态变化。用户不仅能够看到任务是否运行成功，还能够查看任务对应的场景识别结果、决策目标、资源分配摘要和等待原因。对于调度失败或暂时等待的任务，系统能够给出较明确的原因说明，有助于用户理解系统行为，也便于后续测试分析和问题排查。

在工程实现方面，系统采用前后端分离架构，后端按照认证、资源、任务和调度等功能进行模块划分，整体结构较清晰，便于后续扩展。前端界面能够展示资源地图、CPU/GPU 状态、任务详情和调度时间线，具备较好的演示效果。对于毕业设计阶段的系统原型而言，该系统已经具备较完整的资源管理、自动调度、日志反馈和可视化展示能力。

但系统仍然存在一些不足。首先，当前任务执行过程主要采用模拟方式，通过运行时长和时间片机制表示任务推进，并未真正接入实际数字阵硬件或外部计算平台。因此，系统目前更适合作为资源调度管理与验证平台，而不是完整的生产级控制系统。其次，当前调度决策仍以规则和启发式算法为主，虽然具备较好的可解释性和工程稳定性，但在复杂动态环境下的全局最优能力仍有提升空间。后续可以考虑引入基于历史数据的自适应参数调整或智能优化方法。

此外，当前测试规模相对有限。测试环境中资源数量能够覆盖主要调度场景，但尚未进行大规模并发任务、长时间运行和高负载压力测试。因此，系统在更大规模资源池和更复杂任务组合下的稳定性仍需进一步验证。前端可视化方面，系统已经能够展示资

源状态和调度时间线，但对于多任务并发情况下的资源热点统计、任务完成率趋势和调度性能分析等内容仍可继续增强。

综上，本系统在毕业设计阶段已经具备较好的功能完整性和工程可用性，能够支撑数字阵多资源联合调度管理的核心流程。同时，系统在真实设备接入、算法自适应优化、大规模测试和高级可视化分析等方面仍有进一步完善空间。

6.6 本章小结

本章围绕数字阵一体化资源调度管理系统开展测试与结果分析。首先明确了系统测试目的和测试环境，说明了前端、后端、数据库以及天线阵元、CPU、GPU 等模拟资源配置情况；随后设计了低时延任务、多资源均衡任务、GPU 辅助任务、CPU 密集型任务和依赖约束任务等典型测试场景，用于验证系统在不同任务特征下的调度行为；进一步对五种天线调度算法（BFS、Dijkstra、Greedy、Heap、DP）和两种 CPU 调度模式（PACKED、BALANCE）进行了量化性能对比。

测试结果表明，系统能够根据任务特征识别对应场景，并输出较为合理的自动决策目标。在资源分配方面，系统能够完成天线阵元、CPU 和 GPU 的联合规划，其中 GPU 辅助任务能够体现单卡局部性分配，CPU 密集型任务能够体现多节点负载均衡分配，依赖约束任务能够正确进入等待状态并记录阻塞原因。量化对比实验表明，各天线调度算法在执行时间（1.3~60 ms）、可行性（BFS/Dijkstra/DP 仅小规模可行，Greedy/Heap 全规模可行）和解质量上各有侧重，验证了系统自动选择算法策略的合理性；BALANCE 模式在大规模 CPU 需求下将负载方差降低了 99.97%，验证了负载均衡策略的有效性。

同时，本章也分析了系统当前存在的不足，包括任务执行仍以模拟方式为主、测试规模有限、调度策略仍以规则和启发式方法为主，以及前端统计分析能力仍可增强等。总体来看，系统已经能够满足毕业设计阶段对功能完整性、调度正确性和工程可用性的验证要求，并为后续真实设备接入和调度策略优化提供了基础。

第七章 结论与展望

7.1 研究结论

本文围绕数字阵系统中多类异构资源统一管理与联合调度问题，设计并实现了一套基于 Web 的数字阵一体化资源调度管理系统。针对传统资源管理方式中天线阵元、CPU、GPU 等资源分散维护、任务调度依赖人工经验、调度过程缺乏可解释记录等问题，本文从系统总体设计、调度算法实现、任务场景识别、动态反馈和系统测试等方面开展研究，形成了较完整的资源调度管理方案。

在系统设计方面，本文完成了面向数字阵任务的资源、任务和调度过程建模。系统将天线阵元、CPU 节点和 GPU 资源纳入统一管理框架，通过任务表、资源表、分配记录表和调度日志表建立任务需求与资源供给之间的关联关系。同时，系统采用前后端分离架构，后端基于模块化方式实现认证鉴权、资源管理、任务管理和调度控制等功能，前端提供资源地图、任务管理、调度时间线和状态展示等界面，为系统运行和论文实验提供了基础支撑。

在联合调度设计方面，本文围绕天线阵元、CPU 和 GPU 三类资源构建了协同调度流程。针对阵元资源具有阵面分布、空间邻接、复用次数和频率冲突等特点，系统设计了 BFS 快速搜索、Dijkstra 低权重路径搜索、贪心搜索、堆优化搜索和动态规划搜索等多种阵元调度策略，使系统能够在不同任务场景下兼顾搜索速度、分配质量和跨阵面可行性。在计算资源侧，系统实现了紧凑式 CPU 分配、负载均衡式 CPU 分配和 GPU 单卡局部性分配，能够根据任务计算需求和当前资源负载生成相应的计算资源方案。通过“先规划、后落库”的联合调度主循环，系统避免了单类资源提前占用导致整体调度失败的问题，保证了任务状态、资源状态和分配记录之间的一致性。

在任务场景识别与自动决策方面，本文设计了从任务特征到调度目标的映射机制。系统根据任务的阵元需求、CPU 核数需求、GPU 显存需求、优先级、截止时间、依赖关系和互斥关系等信息，识别低时延任务、多资源均衡任务、GPU 辅助任务、高优先级小规模任务、大规模阵元任务、CPU 密集型任务和依赖约束任务等典型场景，并进一步输出快速可行分配、公平均衡分配、单卡局部性分配、低复用高质量分配、跨阵面扩展分配、计算负载均衡分配和依赖阻塞等待等决策目标。该机制使系统由人工选择调度策略转变为根据任务特征自动选择调度路径，提高了系统的自动化程度和可解释性。

在动态反馈与可视化方面，本文设计了调度日志、时间片更新和前端展示机制。系统通过调度日志记录任务进入调度、场景识别、等待原因、资源分配、运行更新和任务完成等关键事件；通过时间片机制更新任务剩余时间和虚拟公平份额，并在任务完成后释放占用资源；通过前端资源地图、CPU/GPU 状态界面和任务调度时间线展示资源状态、任务状态、决策原因和调度过程。上述机制使系统不仅能够给出调度结果，还能够说明任务为何运行、为何等待以及资源如何变化。

在系统测试方面，本文构造了低时延任务、多资源均衡任务、GPU 辅助任务、CPU 密集型任务和依赖约束任务等典型测试场景，并对五种天线调度算法和两种计算资源调度模式进行了量化性能对比。测试结果表明，系统能够根据不同任务特征识别对应场景并输出合理的决策目标；GPU 辅助任务能够触发单卡局部性分配，CPU 密集型任务能够实现多节点负载均衡分配，依赖约束任务能够正确进入等待状态并记录阻塞原因。量化对比实验进一步表明，各天线调度算法在执行时间、可行性和解质量三个维度上各有侧重，系统根据任务特征自动选择算法的策略能够兼顾响应速度与方案质量；BALANCE 模式在 128 核 CPU 密集场景下相比 PACKED 模式将节点负载方差降低了 99.97%。总体来看，本文所实现的系统能够完成数字阵场景下多类资源的统一管理、联合调度和过程反馈，具备较好的工程可行性和毕业设计实现价值。

7.2 不足之处

尽管本文已经完成了数字阵一体化资源调度管理系统的主体设计与实现，并通过典型测试场景验证了系统的基本可行性，但受毕业设计周期、测试条件和系统规模等因素限制，当前系统仍存在一些不足。

首先，系统中的任务执行过程仍以模拟方式为主。当前系统主要通过任务运行时长、剩余时间和时间片机制模拟任务执行过程，尚未与真实数字阵设备、外部计算平台或实际信号处理链路进行深度联动。因此，本文实现的系统更适合作为资源调度管理与验证平台，而不是完整的生产级数字阵控制系统。若要进一步应用到真实工程环境，还需要接入实际设备状态采集、任务执行反馈和异常恢复机制。

其次，系统测试规模仍然有限。本文虽然已经对五种天线调度算法和两种 CPU 调度模式进行了量化性能对比，但测试环境中的资源规模（64 阵元、5 节点）仍相对较小，尚未在更大规模（数百至上千阵元）环境下验证各算法的可扩展性极限，也尚未进行大规模并发任务、长时间运行和高负载压力测试。因此，系统在更复杂任务组合、更大资源池以及持续运行条件下的稳定性和性能表现仍需进一步验证。

再次，当前调度策略主要基于规则和启发式算法实现。系统已经设计了 BFS、Dijkstra、贪心、堆优化、动态规划、负载均衡分配等多种策略，并能够根据任务场景自动选择调度路径，但整体决策过程仍主要依赖预设规则和人工设定的阈值。在资源状态高度动态变化、任务类型更加复杂的场景下，系统还缺少基于历史运行数据的自适应优化能力。

最后，前端可视化展示仍有进一步提升空间。当前系统已经能够展示资源状态、任务详情、调度时间线和决策原因，但对于多任务并发情况下的资源热点分布、任务完成率趋势、调度性能统计和历史运行分析等内容支持还不够充分。后续若要提升系统运维价值，需要进一步增强前端统计分析能力，使系统不仅能够展示单个任务的调度过程，也能够从整体层面对资源使用情况和调度效果进行分析。

7.3 展望

针对当前系统存在的不足，后续可以从真实设备接入、调度策略优化、测试体系完善和可视化能力增强等方面继续开展研究与改进。

首先，可以进一步推动系统与真实数字阵设备或仿真平台的对接。当前系统主要完成了资源建模、任务调度和状态反馈等管理功能，后续若能接入实际阵元状态采集、任务执行平台和信号处理链路，就可以形成从任务提交、资源分配、任务执行到状态回传的完整闭环。这样不仅能够提升系统的工程应用价值，也能够为调度策略提供更加真实的数据来源。

其次，可以在现有启发式调度策略基础上引入更强的自适应优化能力。当前系统主要通过规则和预设阈值完成任务场景识别与调度路径选择，后续可以结合历史调度日志、任务完成时间、资源利用率和等待时间等数据，研究参数自适应调整机制，或者引入遗传算法、强化学习等智能优化方法，用于进一步提升复杂场景下的调度效果。

再次，可以扩大测试规模并完善自动化测试体系。后续可构造更多任务组合，包括高并发任务、长时间运行任务、资源紧张任务和复杂依赖任务等，进一步验证系统在不同压力条件下的稳定性。同时，可以补充单元测试、集成测试和持续集成流程，使系统在后续迭代过程中能够更及时地发现功能异常和调度逻辑问题。

最后，可以继续增强前端可视化和运维分析能力。在现有资源地图、任务详情和调度时间线基础上，后续可以增加资源热点分布、任务完成率趋势、任务等待时间统计、调度策略效果对比等分析页面，使用户能够从整体角度观察系统运行情况。通过进一步完

善可视化能力，系统不仅能够展示单个任务的调度过程，也能够支持对资源利用效率和调度策略效果的长期分析。

总体来看，本文实现的系统已经完成了数字阵一体化资源调度管理的核心功能，为后续真实设备接入、调度策略优化和工程化部署提供了基础。随着系统规模扩大和真实运行数据积累，后续仍有较大的优化和应用空间。

参考文献

- [1] 张浩为, 谢军伟, 师俊朋, 等. 饱和时序下防空相控阵雷达动态优先级调度算法[J/OL]. 北京航空航天大学学报, 2016, 42(12): 2722-2729. DOI:10.13700/j.bh.1001-5965.2015.0779.
- [2] 张浩为, 谢军伟, 胡祺勇, 等. 数字阵列雷达在线交错调度算法[J/OL]. 电子学报, 2019, 47(6): 1260-1266. DOI:10.3969/j.issn.0372-2112.2019.06.011.
- [3] 朱元江, 杨康, 鲍秋香, 等. 数字阵列宽带多波束发射设计与验证[J/OL]. 电子科技大学学报, 2023, 52(6): 818-825. DOI:10.12178/1001-0548.2023077.
- [4] 袁野, 杨剑, 刘辛雨, 等. 基于任务效用最大化的多雷达协同任务规划算法[J/OL]. 雷达学报（中英文）, 2023, 12(3): 550-562. DOI:10.12000/JR23013.
- [5] GHODSIA, ZAHARIA M, HINDMAN B, 等. Dominant Resource Fairness: Fair Allocation of Multiple Resource Types[C]//Proceedings of the 8th USENIX Symposium on Networked Systems Design and Implementation (NSDI 11). Boston, MA: USENIX Association, 2011: 323-336.
- [6] HINDMAN B, KONWINSKI A, ZAHARIA M, 等. Mesos: A Platform for Fine-Grained Resource Sharing in the Data Center[C]//Proceedings of the 8th USENIX Symposium on Networked Systems Design and Implementation (NSDI 11). Boston, MA: USENIX Association, 2011: 295-308.
- [7] BURNS B, GRANT B, OPPENHEIMER D, 等. Borg, Omega, and Kubernetes[J/OL]. Communications of the ACM, 2016, 59(5): 50-57. DOI:10.1145/2890784.
- [8] THE KUBERNETES AUTHORS. Kubernetes Documentation[Z]. 2026.
- [9] SCHEDMD. Slurm Workload Manager Overview[Z]. 2026.
- [10] VOLCANO CONTRIBUTORS. Volcano Documentation[Z]. 2026.
- [11] META OPEN SOURCE. React Documentation[Z]. 2026.
- [12] VITE CONTRIBUTORS. Vite Documentation[Z]. 2026.
- [13] VMWARE, INC. Spring Boot Reference Documentation[Z]. 2026.
- [14] MAILLOUX R J. Phased Array Antenna Handbook[M]. 3 版. Norwood: Artech House, 2018.
- [15] MOORE E F. The Shortest Path Through a Maze[J]. Proceedings of an International Symposium on the Theory of Switching, 1959: 285-292.
- [16] DIJKSTRA E W. A Note on Two Problems in Connexion with Graphs[J/OL]. Numerische Mathematik, 1959, 1: 269-271. DOI:10.1007/BF01386390.
- [17] BELLMAN R. Dynamic Programming[M]. Princeton: Princeton University Press, 1957.
- [18] CORMEN T H, LEISERSON C E, RIVEST R L, 等. Introduction to Algorithms[M]. 3 版. Cambridge: MIT Press, 2009.
- [19] TANENBAUM A S, BOS H. Modern Operating Systems[M]. 4 版. Boston: Pearson, 2015.

- [20] SILBERSCHATZ A, GALVIN P B, GAGNE G. Operating System Concepts[M]. 10 版. Hoboken: Wiley, 2018.
- [21] HOLLAND J H. Adaptation in Natural and Artificial Systems[M]. Ann Arbor: University of Michigan Press, 1975.

致谢

时光匆匆，转眼间本科阶段的学习生活即将结束。回顾毕业设计从选题、系统开发到论文撰写的全过程，我在不断查阅资料、修改方案、调试系统和完善论文的过程中收获了许多。毕业设计不仅是对大学所学知识的一次综合运用，也是对自身分析问题、解决问题和持续改进能力的一次锻炼。在论文即将完成之际，谨向所有给予我帮助和支持的老师、同学和家人表示诚挚的感谢。

首先，衷心感谢我的指导教师胡鹤飞老师。在毕业设计的选题、开题、系统设计和论文撰写过程中，胡老师给予了我耐心的指导和宝贵的建议。从课题方向的确定，到系统功能的梳理，再到论文结构和内容表达的完善，老师都提出了许多具有针对性的意见，使我能够逐步明确研究目标，改进系统设计，并不断提升论文质量。老师严谨认真的治学态度和负责耐心的指导方式，使我受益匪浅。

感谢北京邮电大学信息与通信工程学院各位老师在本科阶段对我的培养。四年的课程学习和实践训练，使我逐步建立了通信工程、软件开发和系统设计等方面的知识基础，也为本次毕业设计的完成提供了重要支撑。在专业课程、实验实践和项目训练中获得的知识与方法，帮助我更好地理解数字阵列系统资源管理、多资源调度以及 Web 系统开发等相关内容。

感谢同学和朋友们在毕业设计过程中的帮助与陪伴。在系统开发、论文撰写和问题讨论过程中，大家给予了我很多建议和鼓励，也在资料查找、技术交流和论文修改等方面提供了支持。正是这些交流和讨论，使我能够从不同角度审视自己的系统设计和论文表达，并不断完善不足之处。

同时，感谢我的家人在大学期间给予我的理解、支持和鼓励。无论是在学习生活中，还是在毕业设计较为紧张的阶段，家人的关心都给了我持续前进的动力，使我能够更加安心地完成学业和论文工作。

最后，感谢在论文完成过程中参考过的相关文献、开源技术和开发工具。它们为本文系统设计与实现提供了重要参考，也使我能够在已有研究和工程实践基础上完成自己的毕业设计。由于本人能力和经验有限，论文和系统中仍难免存在不足之处，恳请各位老师批评指正。

附录 1 社会、健康、安全、法律、文化以及环境等非技术因素影响评估

本文设计并实现的“基于 Web 的数字阵一体化资源调度管理系统”属于面向数字阵任务资源管理与调度过程展示的软件系统。系统主要功能包括天线阵元、CPU、GPU 等资源的统一管理，任务提交与联合调度，调度日志记录以及前端可视化展示等。该系统本身不直接产生物理加工过程，也不直接控制高功率射频设备或危险机械装置，但其应用对象与科研实验平台、计算资源、机房设备和数字阵系统运行管理相关，因此仍需要从社会、健康、安全、法律、文化和环境等非技术因素角度进行综合评估。

从社会影响角度看，本系统通过 Web 平台对多类资源进行统一管理，有助于减少人工分配资源带来的沟通成本和管理误差。在多人共享实验资源或多任务并发运行场景下，系统能够记录任务提交、等待、运行和完成等过程，并给出等待原因和资源分配结果，从而提高资源使用过程的透明度和规范性。对于科研实验室或工程验证平台而言，该系统有助于提升资源管理效率，改善任务调度过程中的公平性和可追踪性，具有一定的工程应用价值。

从健康与安全角度看，本系统本身属于软件管理平台，不直接产生射频辐射、机械运动或化学污染等安全风险。但系统所管理的后台服务器、CPU/GPU 计算节点以及可能接入的数字阵设备，仍然依赖稳定的机房环境和规范的运行管理。若设备长时间高负载运行，可能带来机柜温升、散热不足、供电压力增大等问题。因此，在实际部署过程中，应保证服务器和相关设备处于符合规范的机房环境中，配合散热监控、供电保护、访问控制和定期巡检等措施，降低设备运行风险。

从法律与合规角度看，系统涉及用户登录认证、任务信息、资源状态和调度日志等数据。虽然这些数据通常不属于个人敏感信息，但仍应遵守单位或学校的信息安全管理要求，避免未授权用户访问关键接口或篡改资源状态。本文系统在设计中引入了用户认证、接口鉴权和调度日志记录机制，用于限制未授权访问并保留关键操作记录。若后续系统部署到真实工程环境，还应进一步完善账号权限分级、日志留存、数据备份和异常操作追踪机制，保证系统运行过程符合网络安全和数据管理要求。

从文化与组织协作角度看，传统资源分配往往依赖人工经验和线下沟通，容易出现资源占用情况不透明、任务等待原因不清晰等问题。本系统将任务提交、资源分配和调

度过程转化为可记录、可查询和可展示的流程，有助于形成更加规范、公开和协作化的资源使用方式。对于团队协作而言，系统能够减少因资源争用产生的沟通成本，也有助于提升实验任务安排的公平性和可解释性。

在制造成本方面，本系统作为软件平台，开发过程主要消耗人力、计算机设备和软件开发环境成本，并不涉及额外实体制造过程。系统运行所需硬件主要包括服务器、数据库、前端访问终端以及可能接入的计算节点和数字阵设备。若在已有实验平台或机房环境中部署，新增成本主要体现在服务器运行、系统维护和后续功能扩展上。相比完全依赖人工管理的方式，系统能够提高资源利用效率，减少重复沟通和无效等待，从长期看有助于降低管理成本和资源闲置成本。

在供电问题方面，系统自身运行依赖后台服务器、数据库服务和网络环境。若系统进一步接入 GPU 节点、计算服务器或数字阵实验设备，其供电稳定性会直接影响任务调度和状态记录的可靠性。为避免异常断电导致任务状态不一致、日志丢失或资源占用记录错误，实际部署时应配置稳定供电环境，并结合不间断电源、数据定期备份和异常恢复机制提升系统可靠性。对于高负载计算节点，还应关注功耗变化和散热条件，避免因长期满载运行造成设备风险。

在废弃物处理与循环再利用方面，本系统作为软件系统本身不会直接产生实体废弃物，但其依赖的服务器、显示终端、网络设备和计算卡等硬件在生命周期结束后会形成电子废弃物。相关设备报废时，应按照国家或单位的资产管理和电子废弃物回收流程进行处理，避免随意丢弃造成环境污染。对于仍具备使用价值的服务器、硬盘、显示器和网络设备，可以在满足数据安全清除要求的前提下进行再利用或调拨，提高硬件资源使用寿命。

在节能减排方面，系统通过资源状态展示、任务调度和等待原因记录，有助于减少资源空转和重复占用。在多任务并发场景下，合理的调度策略能够使 CPU、GPU 和天线阵元资源更加有序地被使用，避免任务无效占用资源或长期等待而无人知晓。尤其是在计算资源管理方面，系统通过 CPU 负载均衡和 GPU 单卡局部性分配等策略，有助于提升资源利用效率，减少不必要的计算浪费。后续若进一步加入资源负载统计、低负载休眠和任务批量调度等机制，还可以进一步提升节能效果。

综合来看，本文设计的数字阵一体化资源调度管理系统在社会协作、资源利用、过程透明和工程管理方面具有积极意义。系统本身不直接产生高危健康和环境风险，但其实际部署和运行仍需要关注供电稳定、设备散热、数据安全、权限管理、电子废弃物回

收和节能运行等非技术因素。通过在系统设计和后续部署中充分考虑这些因素，可以提升系统的工程可用性、安全性和可持续发展价值。